

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Исследовательская часть	5
1.1 Обзор предметной области	5
1.2 Общие определения	6
1.3 Фактический язык шаблона	8
1.4 Регулярное представление шаблонов	11
1.4.1 Каскадное произведение автоматов	11
1.4.2 Переход от каскада к ДКА	15
1.5 Сужение шаблона как регулярный язык	18
2 Конструкторская часть	22
2.1 Общая архитектура программного комплекса	22
2.2 Архитектура подсистемы семантического анализатора . . .	23
2.2.1 Вспомогательные подмодули семантического анализатора	23
2.2.2 Проектирование алгоритмов	25
2.3 Пользовательский интерфейс	34
3 Технологическая часть	35
3.1 Обзор используемых технологий	35
3.2 Реализация модуля для работы с абстрактными шаблонами	36
3.3 Реализация модуля для работы с автоматами	38
3.4 Реализация алгоритма обнаружения перекрытий	40
3.4.1 Упрощенная задача	40
3.4.2 Полная задача	42
3.5 Реализация алгоритма извлечения документации	42
3.5.1 Подготовка шаблонов	42
3.5.2 Построение сужений и деревьев префиксов	43
3.5.3 Обязательные инфиксы	44
3.5.4 Эвристики ограничения размера	44
3.6 Схема работы семантического анализатора	44
3.7 Руководство пользователя	46
3.7.1 Получение и сборка	46

3.7.2	Запуск	46
3.7.3	Пример использования	47
4	Аналитическая часть	49
4.1	Тест производительности	52
4.1.1	Конфигурация стенда	52
4.1.2	Методика измерений	53
4.1.3	Результаты	53
4.1.4	Анализ результатов	55
	ЗАКЛЮЧЕНИЕ	57
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	58

АННОТАЦИЯ

Объектом исследования являются программы на языке Рефал, а предметом — статический анализ шаблонов в левых частях функций. В работе рассматриваются две взаимосвязанные задачи: распознавание перекрытий (экранирования) предложений функции и автоматическое извлечение документации, описывающей допустимые формы аргументов для каждого шаблона.

В исследовательской части формализованы языки выражений и абстрактных шаблонов, введено понятие фактического языка шаблона как сужения языка шаблона относительно предшествующих предложений функции. Показано, что языки шаблонов допускают регулярное представление: предложена конструкция каскадного произведения конечных автоматов, позволяющая по вложенной структуре шаблона строить детерминированные конечные автоматы и вычислять сужения с помощью стандартных операций над регулярными языками. На этой основе сформулированы и решены упрощённая и полная задачи распознавания перекрытий, а также разработан алгоритм извлечения документации в виде обязательных префиксов и инфиксов языка сужения.

В конструкторской и технологической частях описана и реализована на языке Rust программная система для диалекта Рефал-5λ. Комплекс включает лексический и синтаксический анализаторы, подсистему семантического анализа с модулями работы с абстрактными шаблонами, конечными автоматами, распознавания перекрытий и генерации документации, а также приложение с интерфейсом командной строки. Пользователь может выполнять упрощённое и полное распознавание перекрытий, а также получать текстовое описание шаблонов функций.

В аналитической части проведена оценка реализации на корпусе из 84 файлов и более 3100 функций проектов MSCP-A и SCP4. Установлено, что подавляющее большинство автоматов сужений имеет не более 20 состояний; сравнение базового и каскадного алгоритмов распознавания перекрытий показало, что каскадный вариант быстрее на типичных «лёгких» функциях, тогда как базовый алгоритм выигрывает на функциях с крупными автоматами и в сумме по корпусу.

ВВЕДЕНИЕ

Основная конструкция в Рефале — сопоставление с образцом. Будучи при этом нетипизированным языком в привычном понимании, Рефал-программы могут демонстрировать потрясающую выразительность. Но с большими возможностями появляется большая ответственность: отсутствие типизации — это источник потенциальных ошибок, а полный статический анализ шаблонов в левых частях функций в общем случае является неразрешимой задачей (в диалектах языка, допускающих использование условий, в условиях могут быть произвольные вычисляемые функции).

Задав фактический язык шаблона, как разность языка шаблона и объединения языков предшествующих шаблонов, можно, анализируя этот язык, узнавать значимые факты о шаблонах функции. Проверяя фактический язык шаблона на пустоту, можно обнаруживать один из видов семантических ошибок: (множественное) перекрытие предложения — ситуация, когда некоторое предложение функции недостижимо при любом значении аргумента. Такие предложения почти всегда появляются в результате ошибки при написании кода, при этом их трудно отловить как на этапе написания программы, так и во время ее выполнения. Первое обусловлено тем, что перекрытия могут быть множественными, т.е. одно предложение экранируется несколькими другими, при этом в функции эти предложения могут идти не подряд и визуально обнаружить такие перекрытия крайне сложно. Второе же обуславливается тем, что экранируемые предложения де-факто являются мертвым кодом, а код, который не исполняется, не порождает ошибок времени выполнения.

Еще одной возможностью является структурный анализ фактического языка шаблона — для него можно, например, извлекать обязательные префиксы, инфиксы и суффиксы, присутствие которых в фактическом языке шаблона не видно «на глаз» при простом просмотре кода функции.

1 Исследовательская часть

1.1 Обзор предметной области

Язык программирования Рефал имеет довольно большую историю, начавшуюся «теоретически» еще в 1966 году, когда В.Ф. Турчин [1] предложил Рефал как метаязык для описания других языков программирования, а «практически» в 1968 г., когда появилась первая реализация Рефала. Рефал имеет множество диалектов, но все они, по сути, являются надстройками над так называемым базисным Рефалом. Базисный Рефал в некотором смысле является самым простым диалектом языка. Программы на базисном Рефале состоят из функций, каждая функция задается парами из шаблона в левой части и выражения в правой.

Единственный тип данных в Рефале — объектное выражение. Это выражение, состоящее из атомарных символов (символ, число) и скобок, при этом скобки правильно вложены. Шаблон в левой части функции — это выражение, в котором, помимо скобок и атомов, могут встречаться переменные. Переменные в шаблоне могут быть трех типов: *e*-переменные, *s*-переменные и *t*-переменные. *e*-переменные могут сопоставляться с любым объектным выражением, *s*-переменные сопоставляются с атомом, *t*-переменные сопоставляются либо с атомом, либо с выражением в скобках. Каждая переменная обладает именем, при этом, если в одном шаблоне есть несколько переменных с одним именем, то при подстановке они заменяются на разные значения.

Основной механизм работы Рефала — сопоставление с образцом и переписывание выражений. При сопоставлении с образцом шаблон в левой части функции сопоставляется с объектным выражением, полученным функцией в качестве аргумента. Если сопоставление прошло успешно, переменным присваиваются конкретные значения, затем вычисляется выражение в правой части. Сопоставление происходит сверху вниз; если входное выражение не сопоставилось ни с одним шаблоном, это приводит к ошибке времени выполнения.

В листинге 1 функция `IsPal` проверяет, является ли переданное ей выражение, состоящее только из атомов, палиндромом.

Листинг 1 — Пример функции на базисном Рефале

```
1  IsPal {
2    s.x e.x s.x = <IsPal e.x>;
3    = True;
4    s.oneChar = True;
5    e.otherwise = False;
6  }
```

Хотя Рефал преимущественно изучается в контексте суперкомпиляции, программы на нем можно исследовать и в контексте статического анализа. С одной стороны, базисный Рефал является чистым функциональным языком, для которого выполнять статический анализ, обычно, легче (по сравнению с императивными языками). С другой стороны, основная конструкция Рефал-программ — присписывание — является ассоциативным, что существенно осложняет задачу анализа.

1.2 Общие определения

Определение 1 (Алфавит). Алфавитом Σ будем называть конечное, если не указано иное, множество символов.

Будем обозначать алфавиты заглавными греческими буквами.

Определение 2 (Связанные со строками понятия). Строкой $w = w_1w_2 \dots w_n$ будем называть конечную последовательность символов из Σ .

Длиной строки w будем называть число n , обозначаемое как $|w|$.

Пустым словом будем называть строку длины 0, обозначаемую как ε .

i -тый символ строки w будем обозначать как $w[i]$.

Строки будем обозначать строчными латинскими буквами.

Определение 3 (Обращение строки). Для строки $w = w_1w_2 \dots w_n$ будем называть ее обращением строку $w^R = w_n \dots w_2w_1$. Для множества строк $W = \{w_1, w_2, \dots\}$ будем называть его обращением множество $W^R = \{w_1^R, w_2^R, \dots\}$.

Определение 4 (Итерация Клини). Множеством строк Σ^* будем называть множество всех конечных строк над алфавитом Σ . $\varepsilon \in \Sigma^*$. Множество $\Sigma^* \setminus \{\varepsilon\}$ обозначим Σ^+ .

Определение 5 (Сокращенная запись конкатенации). Конкатенацией строк $u = u_1u_2 \dots u_n$ и $v = v_1v_2 \dots v_m$ будем называть строку $u \cdot v = u_1u_2 \dots u_nv_1v_2 \dots v_m$.

Если $M \subset \Sigma^*$ — некоторое множество строк и $w \in \Sigma^*$, то будем обозначать как Mw множество $\{mw \mid m \in M\}$. Аналогично для двух множеств строк M_1, M_2 будем обозначать M_1M_2 множество $\{m_1m_2 \mid m_i \in M_i\}$.

Определение 6. Множество $\Sigma^* \setminus W$ будем обозначать $\overline{W}$ и называть дополнением языка W .

Множества строк будем обозначать заглавными латинскими буквами.

Определение 7 (Строковый морфизм). Строковый морфизм $\phi: \Sigma^* \rightarrow \Gamma^*$ — это функция, которая каждому слову $w \in \Sigma^*$ ставит в соответствие слово $\phi(w) \in \Gamma^*$, сохраняющую конкатенацию, т.е. $\forall w_1, w_2 \in \Sigma^* \phi(w_1w_2) = \phi(w_1)\phi(w_2)$. Строковый морфизм однозначно задается набором значений на символах алфавита Σ . Если для некоторого $\sigma \in \Sigma$ не задано явным образом отображение, будем считать, что σ отображается в себя. В таком случае будем опускать этот факт в записи областей определения и значений морфизма.

Все морфизмы в данной работе — строковые, поэтому будем опускать слово "строковый". Будем обозначать морфизмы строчными греческими буквами.

Пример 1. Пусть $\Sigma = \Gamma = \{a, b, c\}$. Морфизм, меняющий местами a и b , можно задать следующим образом: $\phi: \{a, b\} \rightarrow \{a, b\}$, при этом

$$\phi(a) \mapsto b;$$

$$\phi(b) \mapsto a.$$

В таком способе записи неявно задано $\phi(c) = c$.

Определение 8 (Множество обязательных префиксов). Для языка $W \subseteq \Sigma^*$ его множеством обязательных префиксов будем называть множество $P \subseteq \Sigma^*$ такое, что

$$\forall w \in W \exists p \in P \exists v \in \Sigma^* (w = pv).$$

Определение 9 (Обязательный инфикс). Для языка $W \subseteq \Sigma^*$ его обязательным инфиксом будем называть слово $f \in \Sigma^*$ такое, что

$$\forall w \in W \exists v, u \in \Sigma^* (w = vfu).$$

Для $n \in \mathbb{N}$ примем $\overline{1, n} = \{1, 2, \dots, n\}$.

1.3 Фактический язык шаблона

Определение 10 (Язык правильно вложенных слов). Пусть Σ_{call} , Σ_{ret} и Σ_{int} — непесекающиеся алфавиты. Язык правильно вложенных слов $\mathcal{W}$ — это наименьшее множество, которое генерируется следующими конструкторами:

$$\begin{aligned} \varepsilon &\in \mathcal{W} && \text{(пустое слово)} \\ \gamma &\in \mathcal{W} && \text{для каждого } \gamma \in \Sigma_{\text{int}} \quad \text{(внутренний символ)} \\ w_1 \cdot w_2 &\in \mathcal{W} && \text{если } w_1, w_2 \in \mathcal{W} \\ \langle_c w \rangle_r &\in \mathcal{W} && \text{если } w \in \mathcal{W}, \langle_c \in \Sigma_{\text{call}}, \rangle_r \in \Sigma_{\text{ret}} \end{aligned}$$

Выражением будем называть слова языка правильно вложенных слов $\mathcal{W}_{\text{expr}}$, порожденного алфавитами $\Sigma_{\text{call}} = \{(\}\}$, $\Sigma_{\text{ret}} = \{)\}$ и $\Sigma_{\text{int}} = \Delta$, где Δ — счетное множество символов. Абстрактным шаблоном без повторных переменных будем называть слова языка правильно вложенных слов $\mathcal{W}_{\text{pat}}$, порожденного алфавитами $\Sigma_{\text{call}} = \{(\}\}$, $\Sigma_{\text{ret}} = \{)\}$ и $\Sigma_{\text{int}} = \{e, s, t\}$.

Замечание 1. Формальное описание абстрактного шаблона без повторных переменных соответствует таким шаблонам языка Рефал, в которых все переменные имеют различные имена и атомарные символы не встречаются вовсе. Например, для диалекта Рефал-5 λ , поддерживающего безымянные переменные, из абстрактного шаблона можно получить обычный, заменив все символы $e(s, t)$ абстрактного шаблона на символы $e \cdot _ (s \cdot _, t \cdot _)$ шаблона Рефал-5 λ .

Введем также понятие шаблона с пронумерованными переменными: каждому шаблону $p \in \mathcal{W}_{\text{pat}}$ поставим в соответствие слово над алфавитом $\Delta_{\mathbb{N}} = \{(\},)\} \cup \{e, s, t\} \times \mathbb{N}$ такое, что все вхождения символов e, s, t в p в шаблоне с пронумерованными переменными заменяются на буквы из $\{e, s, t\} \times \mathbb{N}$, причем i -тому вхождению символа e в p соответствует буква (e, i) (обозначим e_i). Аналогично для s и t . Скобки $(,)$ остаются на месте. Для шаблона p будем обозначать соответствующий ему шаблон с пронумерованными переменными $p_{\mathbb{N}}$.

Этот формализм нужен лишь затем, чтобы формализовать понятие сопоставления с образцом.

Соответствие переменных множествам их допустимых значений было приведено в обзоре базисного Рефала, теперь же формализуем это соответствие с учетом, что рассматриваются только абстрактные шаблоны без повторных переменных. Будем говорить, что выражение w *допускает сопоставление* с образцом p , если существует строковый морфизм $\phi: \Delta_{\mathbb{N}}^* \rightarrow \mathcal{W}_{\text{expr}}$, действующий следующим образом:

$$\begin{aligned}\phi((s, i)) &\mapsto \gamma \in \Sigma_{\text{int}} \forall i \in \mathbb{N}; \\ \phi((t, i)) &\mapsto \gamma \in \Sigma_{\text{int}} \text{ или } \phi((t, i)) \mapsto (v), v \in \mathcal{W}_{\text{expr}} \forall i \in \mathbb{N},\end{aligned}$$

при этом $\phi(p_{\mathbb{N}}) = w$.

Определение 11 (Язык шаблона). Языком шаблона $p \in \mathcal{W}_{\text{pat}}$ будем называть множество всех выражений $w \in \mathcal{W}_{\text{expr}}$, которые допускают сопоставление с p . Обозначать это множество будем как $\mathcal{L}(p)$.

Каждая функция Рефала задается последовательностью предложений, каждое из которых в свою очередь содержит шаблон в левой части. Для некоторой функции f , содержащей n предложений, будем обозначать i -тый по счету (считая сверху вниз) шаблон предложения как $f.p_i$. Последовательность всех шаблонов функции $\langle f.p_1, f.p_2, \dots, f.p_n \rangle$ будем обозначать как **pat** f .

Не всегда язык шаблона $\mathcal{L}(p)$ совпадает с множеством слов, которые могут сопоставиться с этим шаблоном, находящимся где-то в теле функции. Как правило, языки шаблонов в функции не являются дизъюнктными, а значит часть $\mathcal{L}(p)$ может буквально не доходить до шаблона p , обрабатываясь предложениями выше. Фактическим языком $f.p_i$ будем называть множество

$$\mathcal{L}(f.p_i) \cap \overline{\bigcup_{j \in \overline{1, i-1}} \mathcal{L}(f.p_j)}.$$

Будем называть фактический язык шаблона p сужением шаблона и обозначать как $\mathcal{L}_{\text{fact}}(p)$.

Важным частным случаем сужения является вырождение его в пустой язык. В таком случае имеет смысл говорить о перекрытии (экранировании) шаблона и

соответствующего ему предложения — задаче, родственной анализу избыточности и недостижимости шаблонов при сопоставлении с образцом [2, 3, 4].

Определение 12 (Перекрытие). Предложение i функции f перекрывается (экранируется), если $\mathcal{L}_{\text{fact}}(f.p_i) = \emptyset$.

Определим два варианта задачи распознавания перекрытия:

Определение 13 (Полная задача распознавания перекрытия). Задача распознавания перекрытия в функции f с n предложениями заключается в том, чтобы для каждого шаблона $p_i \in \mathbf{pat} f$, $i \in \overline{1, n}$ найти минимальное по включению множество $I = \{i_1, i_2, \dots, i_k\} \subseteq \overline{1, i-1}$ такое, что p_i перекрывается шаблонами $f.p_{i_1}, f.p_{i_2}, \dots, f.p_{i_k}$, или показать, что такого множества не существует.

Определение 14 (Упрощенная задача распознавания перекрытия). Упрощенная задача распознавания перекрытия в функции f заключается в том, чтобы для каждого шаблона $p_i \in \mathbf{pat} f$ определить, экранируется ли он шаблонами $p_1, p_2, \dots, p_{i-1}$.

Отметим, что решение полной задачи распознавания перекрытия неоднозначно, так как для одного и того же шаблона можно найти несколько минимальных по включению множеств индексов таких, что соответствующие им шаблоны перекрывают искомый. Пример такого случая приведен в листинге 2. Предложение 5 перекрывается предложениями 3 и 4, а также предложением 1. При этом множества $\{3, 4\}$ и $\{1\}$ несравнимы по включению.

Листинг 2 — Пример неоднозначного экранирования предложений Рефал-функции

```

1  F {
2      (t._ t._) e._ = ...;          /* 1 */
3      e._ (s._ e._ s._) e._ = ...;  /* 2 */
4      e._ s._ (t._) = ...;          /* 3 */
5      e._ (e._) (t._) = ...;        /* 4 */
6      (t._ t._) t._ (t._) = ...;    /* 5 */
7  }
```

1.4 Регулярное представление шаблонов

Регулярное представление шаблона представляет особый интерес, поскольку, в сравнении с контекстно-свободными языками (в частности, языками с магазинных автоматов, управляемых входом), коими являются $\mathcal{W}_{\text{expr}}$ и $\mathcal{W}_{\text{pat}}$, регулярные языки более просты для анализа.

1.4.1 Каскадное произведение автоматов

Рассмотрим основную теоретическую конструкцию [5], позволяющую извлекать регулярное представление шаблонов — каскадное произведение автоматов:

Определение 15 (Каскадное произведение автоматов). Пусть $A_1 = \langle Q_1, \Sigma, \delta_1, q_{0_1}, F_1 \rangle$ и $A_2 = \langle Q_2, \Sigma \times Q_1, \delta_2, q_{0_2}, F_2 \rangle$ — детерминированные конечные автоматы. Каскадным произведением $A_1 \circ A_2$ называется ДКА $\langle Q, \Sigma, \delta, q_0, F \rangle$, где

$$\begin{aligned} Q &= Q_1 \times Q_2, \\ q_0 &= (q_{0_1}, q_{0_2}), \\ F &= F_1 \times F_2, \\ \delta((q_1, q_2), a) &= (\delta_1(q_1, a), \delta_2(q_2, (a, q_1))). \end{aligned}$$

Иногда будем просто говорить о каскаде (двух и более автоматов), подразумевая каскадное произведение автоматов.

Определение, данное выше, является вариантом прямого произведения автоматов [6]. Как и многие другие операции над ДКА, использующие прямое произведение (например, пересечение и объединение), каскадное произведение можно описать как систему из двух автоматов, работающих параллельно на одном и том же входном слове. В случае каскада двух автоматов между ними проявляется односторонняя зависимость: оба автомата синхронно читают посимвольно входное слово $w \in \Sigma^*$, при этом первый автомат A работает автономно над алфавитом Σ , а второй автомат, помимо символа $w[i]$, использует текущее (до перехода по $w[i]$) состояние первого автомата q , т.е. работает над алфавитом $\Sigma \times Q_1$.

Основная идея заключается в делегировании распознавания вложенных подшаблонов элементам каскада.

Введем искомый алфавит $\Gamma_0 = \{b, s, (,)\}$. b является метасимволом, соответствующим словам $\mathcal{W}_{\text{expr}} \ni w = (v)$, $v \in \mathcal{W}_{\text{expr}}$, s также является метасимволом, соответствующим словам $\mathcal{W}_{\text{expr}} \ni w = \gamma \in \Sigma_{\text{int}}$. Будем рассматривать регулярные выражения r над алфавитом Γ_0 и ДКА A над этим же алфавитом. Будем обозначать множество слов $\{w \mid w \in \Gamma_0^*\}$, принадлежащие языку $r(A)$, как $R(r)$ ($R(A)$). Отклонение от стандартного обозначения через $\mathcal{L}$ сделано во избежание путаницы: каждое слово языка $r(A)$ в силу смысла символов $\gamma \in \Gamma_0$ само задает некоторый язык слов-выражений из $\mathcal{W}_{\text{expr}}$. Для слова $u \in R(r)$ поставим ему в соответствие слово $u_{\mathbb{N}}$ над $\Gamma_{\mathbb{N}} = \{(,)\} \cup \{b, s\} \times \mathbb{N}$ с пронумерованными переменными, аналогично тому, как это было сделано для шаблонов $p \in \mathcal{W}_{\text{pat}}$ и $p_{\mathbb{N}}$. Будем говорить, что w допускает сопоставление с r , если

$$\exists u \in R(r) \exists \phi: \{b, s\} \times \mathbb{N} \rightarrow \mathcal{W}_{\text{expr}} \mid \phi(u_{\mathbb{N}}) = w,$$

при этом

$$\phi((b, i)) \mapsto (w) \in \mathcal{W}_{\text{expr}} \forall i \in \mathbb{N};$$

$$\phi((s, i)) \mapsto \gamma \in \Sigma_{\text{int}} \forall i \in \mathbb{N};$$

Множество слов $w \in \mathcal{W}_{\text{expr}}$, допускающих сопоставление с r , будем обозначать как $\mathcal{L}(r)$. Любой шаблон $p \in \mathcal{W}_{\text{pat}}$ можно представить в виде регулярного выражения r над алфавитом Γ_0 так, что $\mathcal{L}(p) = \mathcal{L}(r)$. Данный факт следует непосредственно из определения самих e, t, s -символов, ниже представлено соответствие между символами и регулярными выражениями:

- e -символ соответствует выражению $(b|s)^*$;
- t -символ соответствует выражению $(b|s)$;
- s -символ соответствует выражению s .

Введем функцию глубины вложенности на элементах языка правильно вложенных слов:

$$d(w, i) = \begin{cases} 0, & i = 0 \\ d(w, i - 1) + 1, & w[i] \in \Sigma_{\text{call}} \\ d(w, i - 1) - 1, & w[i] \in \Sigma_{\text{ret}} \\ d(w, i - 1), & w[i] \in \Sigma_{\text{int}} \end{cases}$$

Тогда максимальным уровнем вложенности некоторого слова w будет

$$\max d(w) = \max_{i \in \overline{1, |w|}} d(w[i]).$$

Подшаблоном p уровня вложенности k (или просто k -подшаблоном) будем называть подслово $w[i : j]$ такое, что

$$d(w, i) = k \wedge d(w, j) = k - 1 \wedge \forall m \in \overline{i + 1, j - 1} \, d(w, m) \geq k - 1.$$

Обозначим множество всех k -подшаблонов p как $\mathbf{subpat}_k(p)$.

Рассмотрим теперь теоретическую конструкцию каскада по данному шаблону p . Сначала для k , равного максимальной глубине вложенности p , построим регулярные выражения r_i над алфавитом Γ_0 по правилам, описанным выше, для всех k -подшаблонов p_i ; далее построим соответствующие ДКА A_i .

Рассмотрим ДКА-объединение A всех A_i с размеченными состояниями: каждое финальное состояние f автомата A помечается множеством $\alpha \subset \mathbf{subpat}_k(p)$, если все пути в автомате A до состояния f принадлежат языкам регулярных выражений, соответствующим k -подшаблонам $p_j \in \alpha$. Чтобы автомат корректно работал только с k -шаблонами, строится композиция автомата A с автоматом-лифтом на уровень скобочной вложенности k . Начальным состоянием A становится начальное состояние лифта, последний «этаж» лифта совмещается с бывшим начальным состоянием A . Пример автомата-лифта для уровня 2 приведен на рисунке 1.

Полученный автомат A является управляющим автоматом каскада. Если $k > 1$, то следующий элемент каскада будет управляться символом входного слова и состоянием A . Для построения управляемого автомата B необходимо

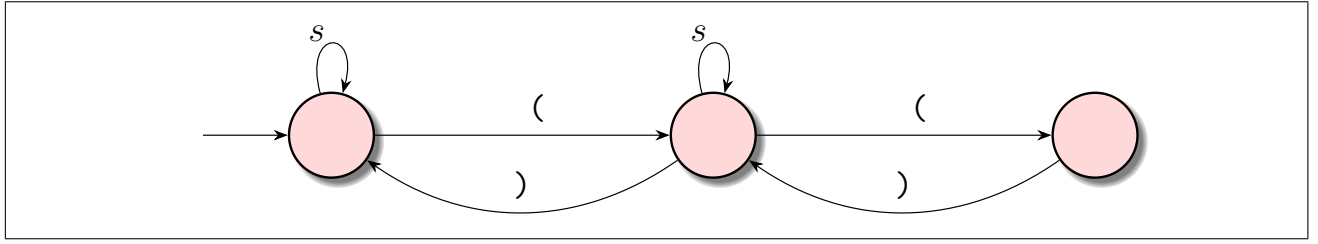


Рис. 1 — Автомат-лифт для уровня вложенности 2.

рассмотреть множество $\mathbf{subpat}_{k-1}$, его элементы включают себя k -шаблоны в качестве подшаблонов. Для принятия решения о переходе по этим k -шаблонам автомат B использует состояния автомата A , аннотации финальных состояний A позволяют автомату B получать необходимую информацию о том, какие шаблоны прочитаны A .

Итак, конструкция выше задает пару автоматов, каскадное произведение которых $A \circ B$ распознает $k - 1$ -шаблоны. Повторяя процедуру выше для $k, k - 1, \dots, 1$ -шаблонов, получаем каскад $A_k \circ A_{k-1} \circ \dots \circ A_0$ с аннотациями 1-шаблонов.

При решении задачи экранирования и задачи порождения документации необходимо различать множество $\bigcup_i \mathbf{subpat}_k(p_i)$ всех k -подшаблонов всех шаблонов некоторой функции f . Конструкция выше тривиально обобщается на несколько шаблонов: для всех k вместо $\mathbf{subpat}_k(p)$ конкретного шаблона рассматривается объединенное множество $\bigcup_i \mathbf{subpat}_k(p_i)$. Теперь из ДКА 0-шаблонов можно получить ДКА каждого шаблона по отдельности, оставив финальными только те состояния, в аннотациях которых присутствует данный шаблон.

Рассмотрим пример функции, приведенной в листинге 3.

Листинг 3 — Пример последовательности образцов с вложенными скобочными уровнями

```

1  f {
2    t.first (e.x (s.y) s.z) = ...;
3    ((t.x) e.y s.z) s.last = ...;
4  }
```

Для примера в листинге 3 шаблонами уровня вложенности 2 будут s , t ; уровня 1 — $e(s)s$, $(t)es$; без скобочной вложенности — $t(e(s)s)$, $((t)es)s$. Обозначения шаблонов см. в таблице 1.

Таблица 1 — Шаблоны примера (листинг 3) и обозначения в аннотациях состояний ДКА

Шаблон	Уровень вложенности	Обозначение
s	2	1
t	2	2
e (s) s	1	I
(t) e s	1	II
t (e (s) s)	0	A
((t) e s) s	0	B

На рисунках 2, 3 и 4 представлены элементы каскада автоматов. Состояния лифта выделены красным, состояния аннотированного ДКА-объединения имеют пунктирную границу. Злоупотребляя нотацией, будем полагать, что во всех состояниях младших аннотированных ДКА для k -шаблонов, у которых нет явного перехода по открывающей скобке, находится лифт на $d - k$ уровней, где d — глубина наиболее вложенного подшаблона в функции. Такая конструкция необходима, чтобы младший автомат выходил из ожидания по корректной закрывающей скобке. Кроме того, из всех видимых состояний, из которых нет перехода по закрывающей скобке, неявно присутствует переход в состояние-предпоследний «этаж» лифта. Аннотированные состояния помечены финальными с целью подчеркнуть значимость сигналов, порождаемых этими состояниями, для следующего автомата в каскаде — для младшего автомата наибольший интерес представляют именно аннотированные состояния. Большинство автоматов имеют полную функцию переходов, но там, где она неполна, предполагаем существование ловушки — все отсутствующие переходы ведут в нее, выйти из ловушки можно только по закрывающей скобке в состояние лифта.

1.4.2 Переход от каскада к ДКА

Конструкция выше является теоретическим обоснованием подхода, описанного ниже. Покажем, что от явного построения каскада можно избавиться вовсе и обойтись лишь построением аннотированного ДКА-объединения для подшаблонов всех уровней вложенности.

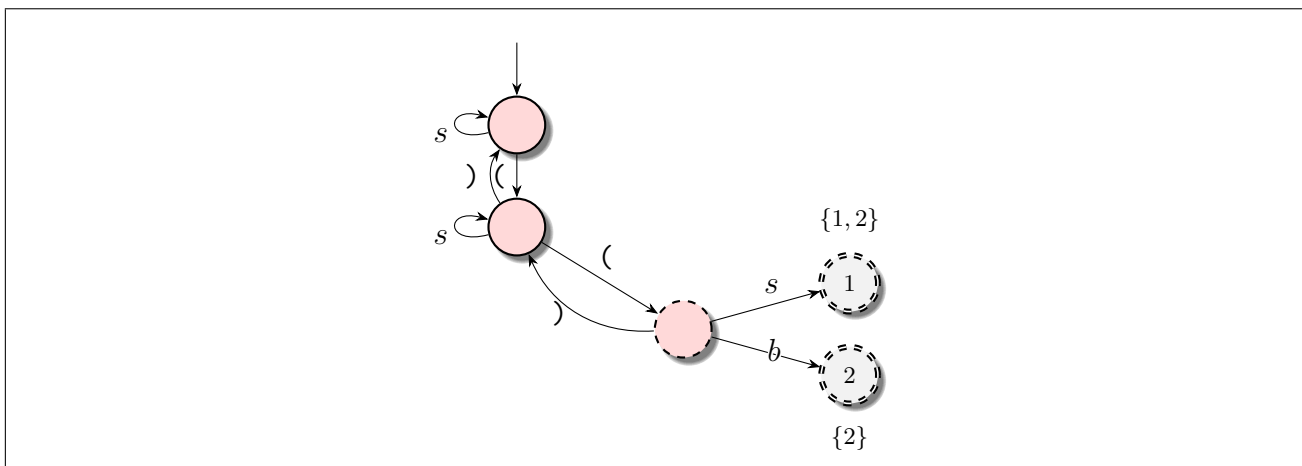


Рис. 2 — Аннотированный распознаватель для шаблонов уровня вложенности 2.

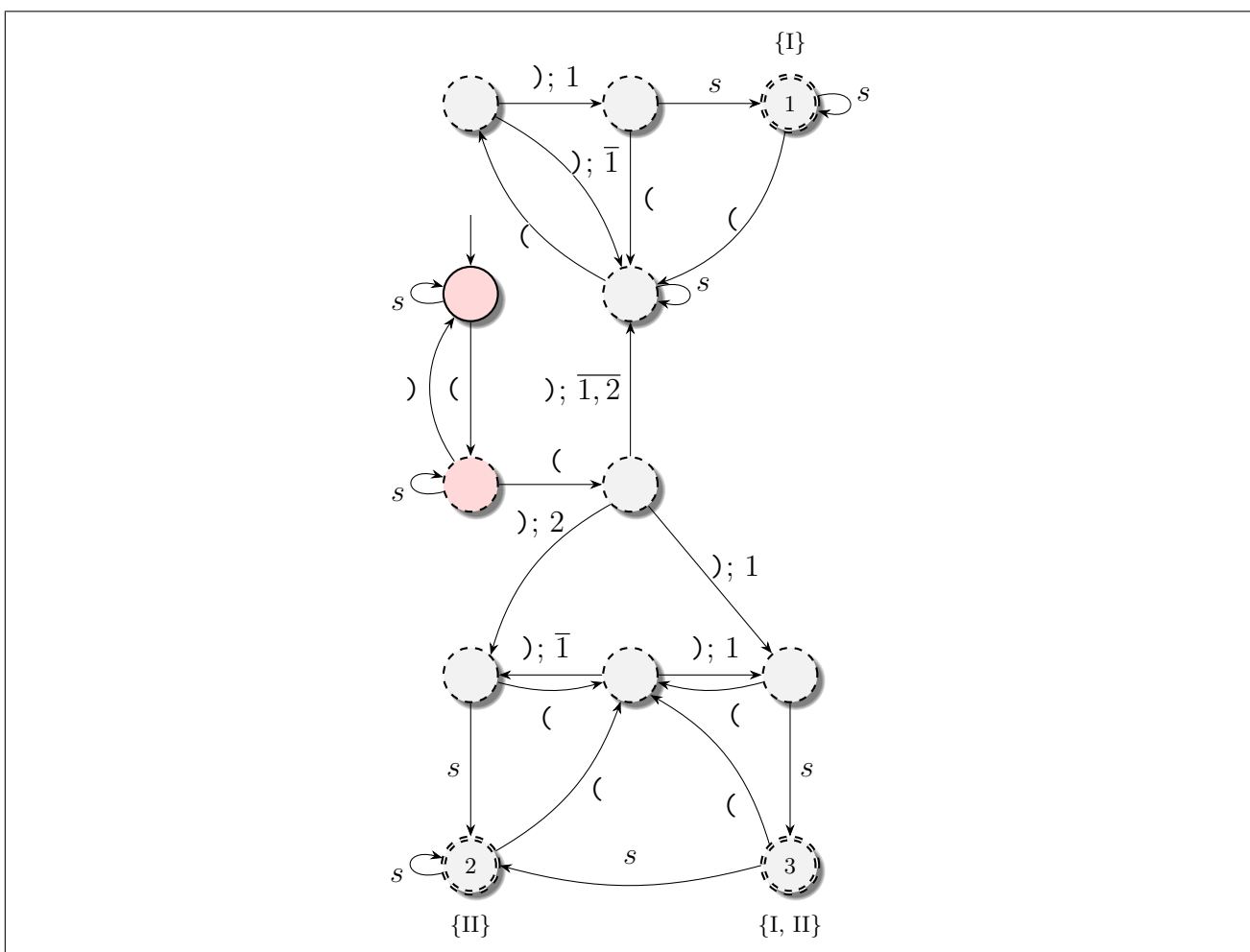


Рис. 3 — Пример аннотированного ДКА для 1-шаблонов.

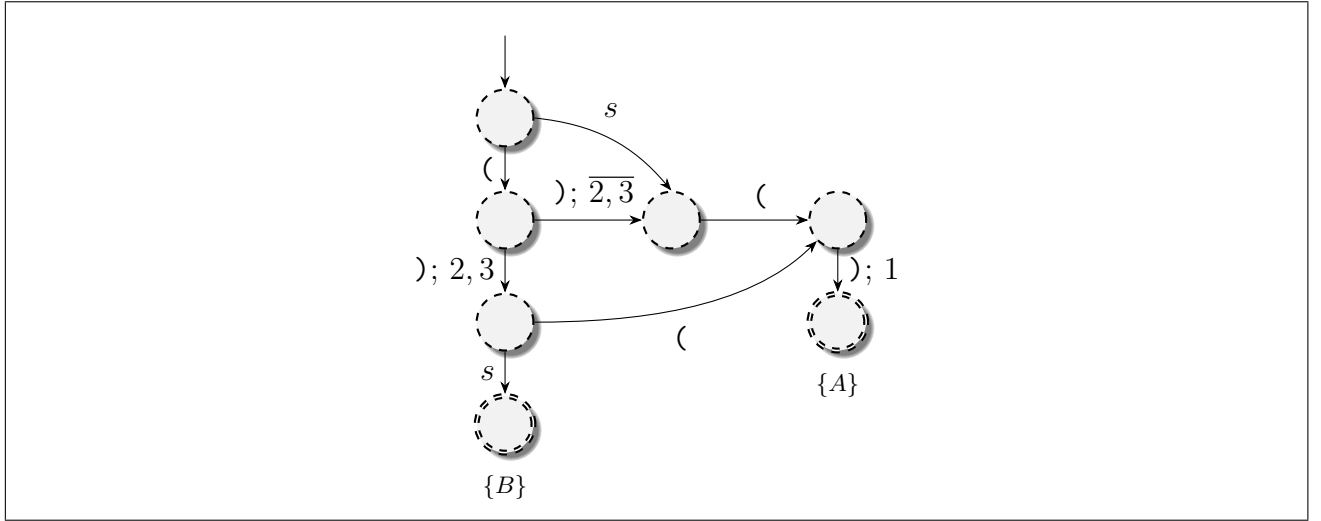


Рис. 4 — Последний элемент каскада уровня 0.

Рассмотрим структуру любого управляемого элемента каскада. В любом состоянии после перехода по открывающей скобке управляемый автомат "ожидает" закрывающую скобку нужного уровня, фактически игнорируя все символы, поступающие на вход до нее. Этот факт потребует в дальнейшем.

Введем на множестве состояний Q аннотированного автомата отношение эквивалентности $\sim_\alpha$, для $q_1, q_2 \in Q$ имеем $q_1 \sim_\alpha q_2$ тогда, и только тогда, когда аннотация q_1 совпадает с q_2 . Будем называть элементы этого фактора $b_0, b_1, \dots, b_n$.

Заметим, что все состояния элемента каскада с одинаковыми аннотациями неразличимы с точки зрения элемента младшего уровня, а также то, что для состояний с уникальной аннотацией соответствующие языки путей дизъюнкты относительно языков путей к другим аннотациям. Выразаясь немного неформально, язык b_i не пересекается с языками b_j , $j \neq i$.

Воспользовавшись наблюдениями выше, можно свести каскад к построению отдельных ДКА в различных алфавитах по следующим правилам:

1. Первый управляющий автомат $A_k = \langle Q, q_0, F, \delta \rangle$ над $\{s, b\}$ строится как и ранее.
2. Строится фактор-множество $Q / \sim_\alpha$, при этом к нему добавляется элемент $\emptyset$, если в A_k есть хоть одно непринимаящее полезное состояние.
3. Младший элемент каскада строится над алфавитом $s, b_0, b_1, \dots, b_n$ следующим образом — по шаблону все так же можно построить регулярное выражение, но символ b теперь заменяется набором $b_0, b_1, \dots, b_n$. По регулярному выражению построить ДКА.

4. Если еще не построен ДКА для 0-шаблонов, перейти к шагу 2, иначе завершить построение.

Таким образом, в данной конструкции бремя различия несовпадающих вложенных подшаблонов ложится на алфавит. Конечным результатом алгоритма, следующего правилам выше, будет ДКА искомого шаблона в некотором алфавите $s, b_0, b_1, \dots, b_n$.

На рисунках 5, 6 и 7 представлен пример упрощенной конструкции ДКА справа рядом с полным каскадом слева. Рассматривается функция f из примера в листинге 3, обозначения шаблонов см. в таблице 1. В каскаде синим цветом выделены состояния, «оставшиеся» в упрощенной конструкции.

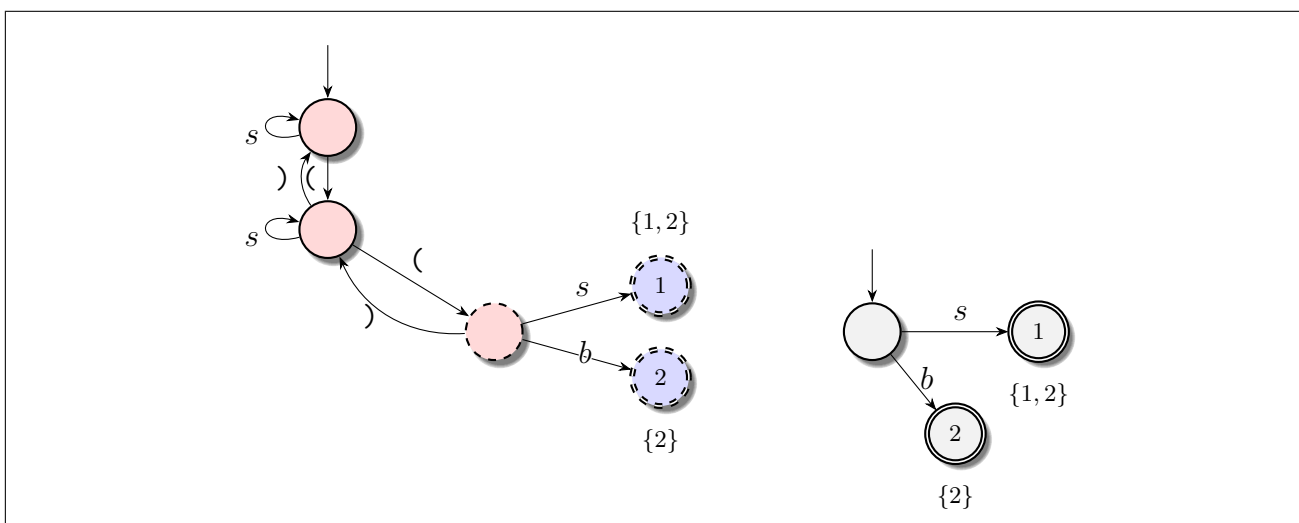


Рис. 5 — Аннотированный распознаватель для шаблонов уровня вложенности 2 (слева). Справа — тот же распознаватель без лифта над алфавитом $\{b, s\}$.

Заметим, что по построению общее число аннотаций на каждого уровне относительно числа k -шаблонов n есть $O(2^n)$. Именно этим обусловлен отказ от рассмотрения шаблонов, содержащих атомарные символы — в шаблонах с высоким уровнем вложенности различие констант может привести к очень сильному разрастанию алфавита.

1.5 Сужение шаблона как регулярный язык

Используя конструкцию ДКА из раздела выше для построения автоматов по набору шаблонов в функции, и используя базовые операции над этими ДКА, можно построить сужения каждого шаблона.

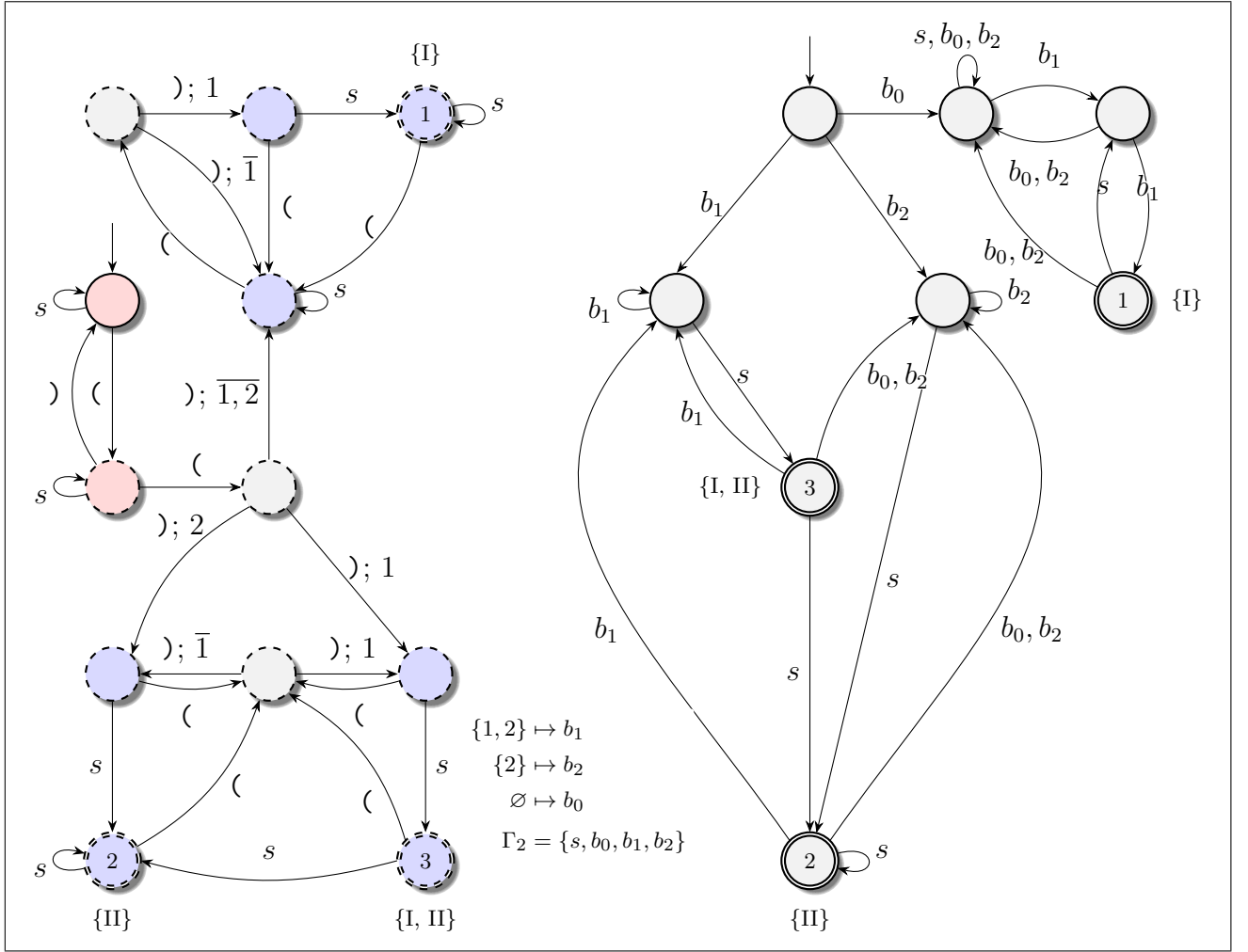


Рис. 6 — Пример аннотированного ДКА (слева) для 1-шаблонов. Справа — упрощенный распознаватель над алфавитом $\{s\} \cup Q_2 / \sim_\alpha$.

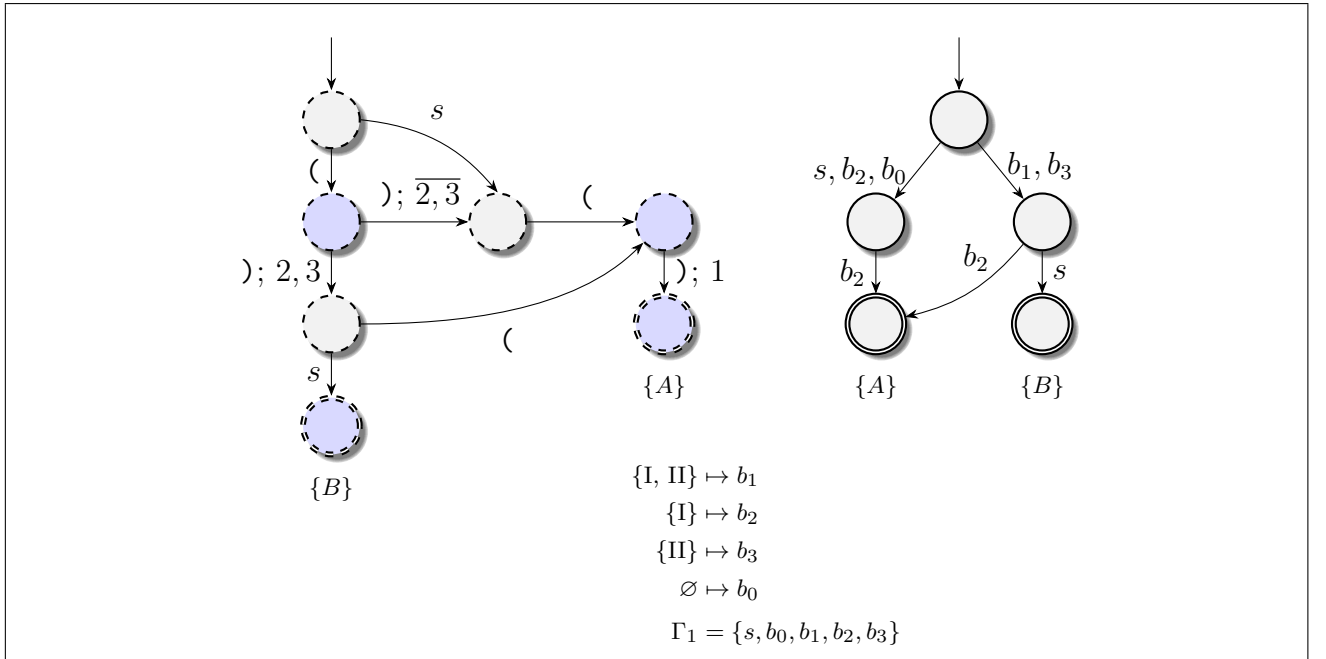


Рис. 7 — Последний элемент каскада уровня 0 (слева). Справа — упрощенный автомат-распознаватель над алфавитом $\{s\} \cup Q_1 / \sim_\alpha$.

Упрощенная задача экранирования для функции f теперь тривиально решается проверкой $\mathcal{L}_{\text{fact}}(f.p_i) = \emptyset$ для всех $p_i \in \mathbf{pat} f$.

Для полного решения задачи экранирования для функции f , решим сначала упрощенный вариант задачи, а потом для каждого экранируемого шаблона p_i рассмотрим копию функции f , из которой поочередно будем пытаться удалять шаблоны $p_{i-1}, p_{i-2}, \dots, p_1$. Если при удалении очередного шаблона $f.p_j$ и последующем решении упрощенной задачи экранирования для оставшейся части функции f результат будет пустым множеством, то шаблон p_j участвует в экранировании шаблона p_i и остается в функции. В противном случае он удаляется.

Теперь же рассмотрим сужения в контексте извлечения документации о шаблоне.

Рассмотрим пример гипотетической функции, приведенной в листинге 4.

Листинг 4 — Простой пример уточняемого шаблона в определении функции (в диалекте Рефал-5λ)

```

1 AnalyzeData {
2   t.node /* empty */ = <AnalyzeData <AppendData t.node>>;
3   t.node t.data, t.data : {
4     ...
5   };
6   e.otherwise = <Error>;
7 }
```

В данном примере очевидно, что шаблон третьего предложения $AnalyzeData.p_3$ можно уточнить:

$$e.otherwise \xrightarrow{\sim} t.1 \ t.2 \ t.3 \ e.4;$$

т.е. сужение шаблона $e.otherwise$ совпадает с $\mathcal{L}(t.1 \ t.2 \ t.3)$.

Менее очевидный пример представлен в листинге 5. Для последнего шаблона можно вывести, что все слова, сопоставляющиеся с этим шаблоном, или начинаются с пустых скобок $()$, либо со слов, удовлетворяющих шаблону $((t.1 \ e.2) e.3)$.

Листинг 5 — Нетривиальный пример уточняемого шаблона в определении функции

```
1  h {  
2    () t.1 = ...;  
3    (e.1 () e.3) e.4 = ...;  
4    (s.1 e.1) e.2 = ...;  
5    (e.1) e.2 s.1 e.3 = ...;  
6    (e.1) e.2 = ...;  
7  }
```

Обходя в ширину ДКА фактического языка 0-шаблонов можно строить дерево обязательных префиксов. Помимо этого, можно извлекать обязательные инфиксы. Это простое наблюдение нуждается в аккуратной реализации: в самом наглядном представлении обязательных префиксов — перечислении — может оказаться слишком много элементов. Из-за сложности нахождения всех обязательных инфиксов для данного ДКА, разумно использовать аппроксимацию. ДКА можно представить как граф, вершины которого есть состояния ДКА, а ребра — переходы между состояниями (переходы между одинаковыми состояниями по разным символам схлопываются в одно ребро, которое помечается всеми этими символами). С помощью алгоритма Тарьяна [7] в графе можно найти мосты. Дополнительно проверив, что ни одно финальное состояние не достигается без посещения «начала» моста, можно получить некоторые обязательные инфиксы языка ДКА.

2 Конструкторская часть

2.1 Общая архитектура программного комплекса

Было решено работать с диалектом Рефал-5λ, поскольку этот диалект, во-первых, является точным надмножеством Рефала-5, а во-вторых является современной реализацией Рефала, знакомой автору.

Рационально поделить программный комплекс на две части — библиотеку, содержащую всю логику работы с шаблонами, автоматами, алгоритмами и т.п., и приложение с интерфейсом командной строки, использующее эту библиотеку.

Устройство самой библиотеки следует классической архитектуре большинства фронтендов компиляторов [8]. Таким образом, библиотека делится на три модуля:

- модуль `lexer` — модуль, отвечающий за лексический анализ;
- модуль `parser` — модуль, отвечающий за синтаксический анализ;
- модуль `sema` — модуль, отвечающий за семантический анализ.

Модуль `sema` единственный предоставляет публичные функции, доступные для вызова извне. Сами модули взаимодействуют друг с другом весьма ограничено, сохраняя свою внутреннюю логику закрытой. Схема взаимодействия модулей и пользователя представлена на рисунке 8.

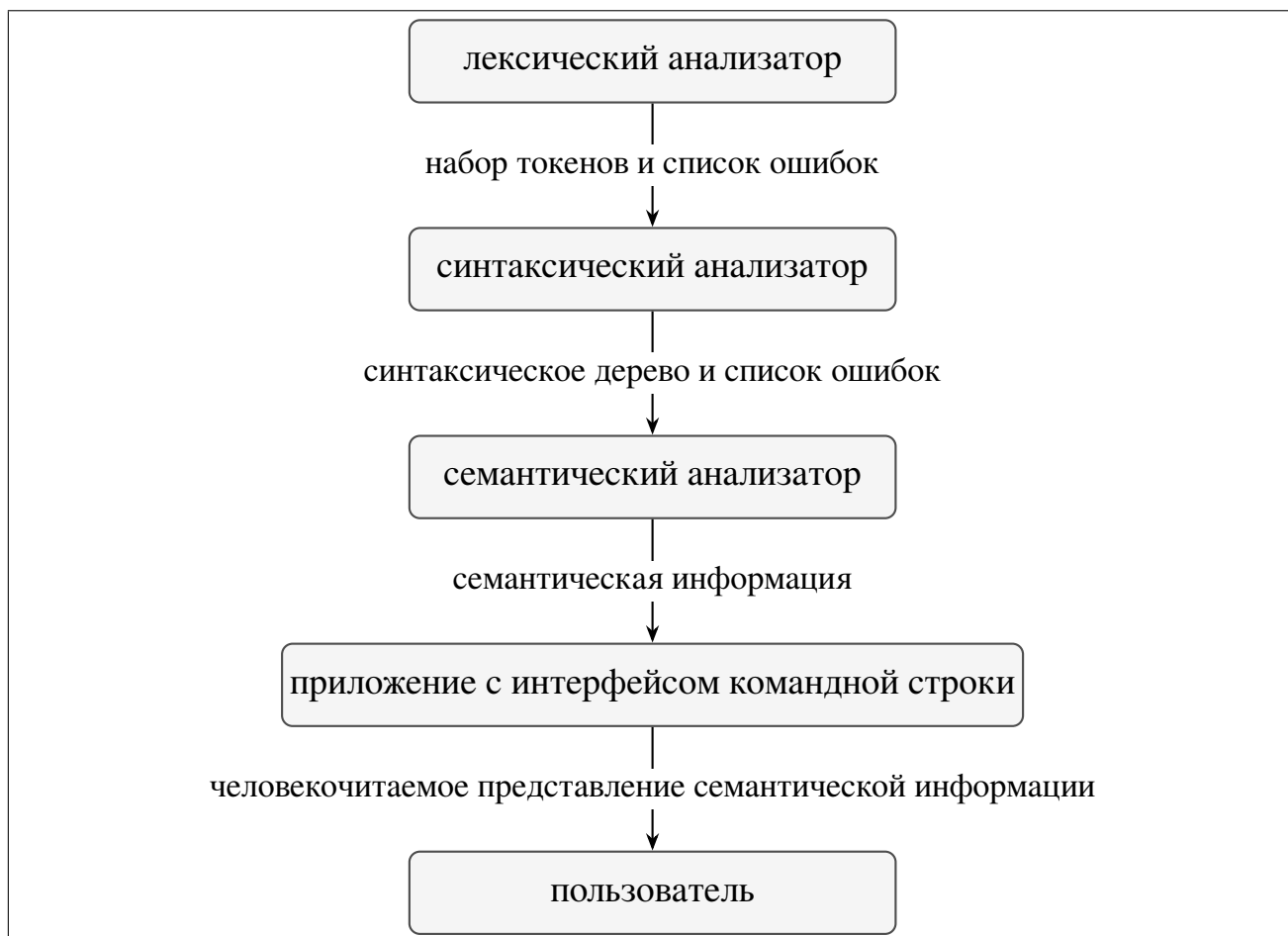


Рис. 8 — Схема взаимодействия модулей программного комплекса.

Лексический и синтаксический анализаторы не представляют особого интереса — Рефал-5 имеет сравнительно простую лексическую структуру, а грамматика языка [9] допускает реализацию синтаксического анализа рекурсивным спуском [8]. Так как основная цель работы — написание семантического анализатора, лексический и синтаксический анализаторы реализованы довольно просто, и не имеют продвинутых способов восстановления после ошибок.

2.2 Архитектура подсистемы семантического анализатора

2.2.1 Вспомогательные подмодули семантического анализатора

Семантический анализатор, будучи самым крупным модулем библиотеки, содержит в себе несколько подмодулей, выполняющих различные задачи.

Первым таким подмодулем является модуль automata, отвечающий за работу с автоматами и за способ хранения ДКА и НКА в памяти. При разработке

прототипа приложения и его тестировании было установлено, что в абсолютном большинстве случаев автоматы имеют размер до десяти тысяч состояний, а размер алфавита не превышает одной тысячи. В связи с этим в итоге было выбрано сжатое разреженное строковое представление для исходящих и входящих ребер. Пример такого представления только для исходящих ребер приведен на рисунке 9 (для входящих ребер схема аналогична). Отслеживать входящие ребра легко при построении ДКА, в дальнейшем это дает выигрыш при реализации алгоритмов минимизации и удаления бесполезных состояний.

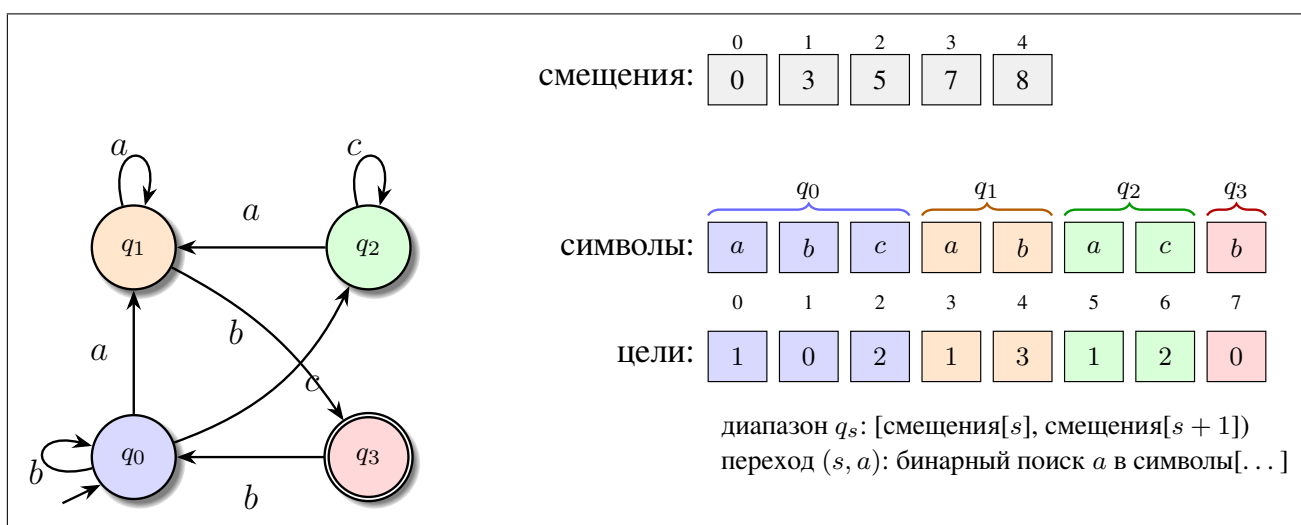


Рис. 9 — Представления ДКА в памяти

В числе операций на ДКА, реализованных в модуле, присутствуют:

- объединение;
- аннотированное объединение;
- пересечение;
- конкатенация (через прямое произведение);
- конкатенация (через построение НКА);
- дополнение;
- проверка языков автоматов на включение;
- минимизация;
- удаление бесполезных состояний.

Необходимость двух вариантов конкатенации ДКА будет обусловлена в следующем разделе. НКА в модуле нужны лишь для того, чтобы стать результатом конкатенации ДКА и сразу же быть детерминизированными.

Второй подмодуль `abstract_patterns` позволяет из конкретных узлов синтаксического дерева строить абстрактные шаблоны, эквивалентные шаблонам из

$\mathcal{W}_{\text{pat}}$. Наличие такого модуля значительно облегчает дальнейшую обработку шаблонов. Ключевой функцией в модуле является функция получения из узла синтаксического дерева определения функции набора абстрактных шаблонов и сопутствующей информации; наличие этой информации обусловлено необходимостью различать шаблоны, изначально удовлетворяющие определению шаблонов из $\mathcal{W}_{\text{pat}}$, от остальных. Модуль в том числе несет ответственность за преобразование всех шаблонов к виду, описанному в подразделе 1.3 исследовательской части. Информация об абстрактном шаблоне включает в себя следующие факты:

- были ли в искомом шаблоне повторные переменные;
- были ли в искомом шаблоне атомарные символы;
- были ли в предложении функции, соответствующем искомому шаблону, условия.

2.2.2 Проектирование алгоритмов

Центральным алгоритмом является построение ДКА абстрактного шаблона. Псевдокод этого алгоритма приведен в листинге 1.

Листинг 1 Построение ДКА абстрактного шаблона

Require: Шаблоны $p_1, p_2, \dots, p_n$

Ensure: ДКА $A_1, A_2, \dots, A_n$ для шаблонов $p_1, p_2, \dots, p_n$

```
1:  $d_{\max} \leftarrow$  максимальная глубина подшаблонов
2:  $\Gamma \leftarrow \{s, b\}$ 
3: for  $d \in \langle d_{\max}, d_{\max} - 1, \dots, 1 \rangle$  do
4:    $S_d \leftarrow \bigcup_{i=1}^n \text{subpat}_d(p_i)$   $\triangleright$  Инвариант цикла —  $S_d$  не содержит скобок
5:   foreach  $p \in S_d$  do
6:     Построить ДКА  $A_p$  над  $\Gamma$  через конкатенацию ДКА-фрагментов
7:   end foreach
8:    $A \leftarrow \text{АннотированноеОбъединение}(\{A_p : p \in S_d\})$ 
9:    $\Gamma \leftarrow \{s\} \cup Q(A)/\sim_\alpha$ 
10:  if в  $A$  есть полезное непринимаяющее состояние then
11:     $\Gamma \leftarrow \Gamma \cup \{\emptyset\}$ 
12:  end if
13:  foreach  $pat \in \{p_1, \dots, p_n\}$  do
14:    foreach  $p \in S_d$  do
15:      Заменить вхождение  $p$  в  $pat$  на регулярное выражение  $r = (I_1 \mid$ 
       $I_2 \mid \dots \mid I_m)$ , где  $I_i \in \Gamma$  входит в альтернативу тогда и только тогда, когда
       $p \in \alpha(I_i)$ 
16:    end foreach
17:  end foreach
18: end for
19: foreach  $p_i \in \{p_1, \dots, p_n\}$  do
20:   Построить ДКА  $A_i$  над  $\Gamma$  через конкатенацию ДКА-фрагментов
21: end foreach
22: return  $A_1, A_2, \dots, A_n$ 
```

В алгоритме построения ДКА под ДКА-фрагментами понимаются небольшие ДКА, построение которых занимает $O(|\Gamma|)$ времени. Так, для t автомат будет

содержать два состояния: непринимающее начальное и принимающее, причем из первого во второе есть переходы по всем символам алфавита. Для e автомат будет содержать одно состояние: принимающее, из которого есть петли по всем символам алфавита. Для s автомат будет содержать два состояния: принимающее и непринимающее начальное, из которого есть переход по символу s . Для альтернативы $(\gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_m)$ автомат будет содержать два состояния (начальное непринимающее и принимающее), между которыми есть переходы по $\gamma_1, \gamma_2, \dots, \gamma_m$.

Другим важным алгоритмом, эксплуатирующим предыдущий, является построение сужения шаблона в листинге 2.

Листинг 2 Построение сужений шаблонов функции

Require: Шаблоны $p_1, p_2, \dots, p_n$ функции в порядке сверху вниз

Ensure: ДКА $F_1, F_2, \dots, F_n$ такие, что $\mathcal{L}(F_i) = \mathcal{L}_{\text{fact}}(p_i)$

```

1:  $(A_1, \dots, A_n) \leftarrow \text{ПостроениеДКА}(p_1, \dots, p_n)$  ▷ см. листинг 1
2:  $A_{\text{acc}} \leftarrow \text{ДКА с пустым языком над алфавитом } A_1$ 
3: foreach  $i \in \overline{1, n}$  do
4:    $F_i \leftarrow A_i \cap \overline{A_{\text{acc}}}$ 
5:    $A_{\text{acc}} \leftarrow A_{\text{acc}} \cup A_i$ 
6: end foreach
7: return  $F_1, F_2, \dots, F_n$ 

```

Имея ДКА абстрактного шаблона, можно использовать их для решения задачи распознавания перекрытий и получения документации.

Отметим, что сужать фактический язык шаблона могут лишь те шаблоны, язык которых «не искажен» различным образом. Поэтому в алгоритме в сопоставлении фактического языка участвуют только те шаблоны, которые изначально удовлетворяли определению шаблонов языка $\mathcal{W}_{\text{pat}}$ (абстрактизация конкретного шаблона всегда расширяет его язык). Еще одно требование — в предложениях шаблонов, участвующих в сужении, не должно быть условий (условия обычно искажают язык шаблона p в том смысле, что множество распознаваемых им слов из-за условия является лишь подмножеством $\mathcal{L}(p)$). Именно по этой причине необходимо сохранять информацию об условиях в абстрактных шаблонах.

Алгоритм решения упрощенной задачи распознавания перекрытий представлен в листинге 3.

Листинг 3 Упрощенная задача распознавания перекрытий

Require: Шаблоны $p_1, p_2, \dots, p_n$ функции в порядке сверху вниз

Ensure: $I \subseteq \overline{1, n}$ — множество индексов перекрываемых предложений

```
1:  $(F_1, \dots, F_n) \leftarrow \text{ПостроениеСужений}(p_1, \dots, p_n)$  ▷ листинг 2  
2:  $I \leftarrow \emptyset$   
3: foreach  $i \in \overline{1, n}$  do  
4:   if  $\mathcal{L}(F_i) = \emptyset$  then  
5:      $I \leftarrow I \cup \{i\}$   
6:   end if  
7: end foreach  
8: return  $I$ 
```

Алгоритм решения полной задачи распознавания перекрытий представлен в листинге 4.

Листинг 4 Полная задача распознавания перекрытий

Require: Функция f с шаблонами $p_1, p_2, \dots, p_n$

Ensure: Для каждого $i \in I$ — минимальное по включению множество $I_i \subseteq \overline{1, i-1}$

```
1:  $I \leftarrow \text{РаспознаваниеПерекрытий}(p_1, \dots, p_n)$  ▷ листинг 3
2:  $\mathcal{I} \leftarrow \emptyset$ 
3: foreach  $i \in I$  do
4:    $S \leftarrow \langle f.p_1, f.p_2, \dots, f.p_{i-1} \rangle$ 
5:   for  $j \in \langle i-1, i-2, \dots, 1 \rangle$  do
6:      $S' \leftarrow S$  без шаблона  $f.p_j$ 
7:      $(F_1, F_2, \dots, F_{n'}) \leftarrow \text{ПостроениеСужений}(S')$  ▷ листинг 2
8:     if  $\mathcal{L}(F_{n'}) = \emptyset$  then
9:        $S \leftarrow S'$ 
10:    end if
11:  end for
12:   $I_i \leftarrow \{j \in \overline{1, i-1} \mid p_j \in S\}$ 
13: end foreach
```

Обратим внимание, что при работе с реальными программами целесообразно сначала находить решение упрощенной задачи, так как экранирование не является часто встречающимся явлением, а потом уже для функций, в которых встречается экранирование, находить решение полной задачи. Обусловлено это сложностью построения ДКА (в том числе промежуточных) с большим алфавитом (размер которого может в худшем случае быть экспоненциальным от количества различных шаблонов данного уровня вложенности). Операции над ДКА, необходимые для построения сужений, выполняются гораздо быстрее, но, тем не менее, если для решения классической задачи требуется порядка $O(N)$ операций для некоторого N , то для решения полной задачи сужения требуется уже порядка $O(N^2)$ операций.

Если сужение шаблона непусто, можно извлекать некоторые факты о нем. Одним из вариантов является анализ множества обязательных префиксов и обязательных инфиксов.

Дерево обязательных префиксов, построенное для ДКА шаблона 0-го уровня вложенности, может содержать символы $b_1, b_2, \dots, b_n$. Для человека такие символы никакой смысловой нагрузки почти не несут, поэтому для каждого символа строится свое дерево обязательных префиксов. Это продолжается до тех пор, пока не будет получено дерево над алфавитом $\{s, b\}$. Однако в результате склейки таких деревьев результирующее дерево может оказаться слишком большим. Для баланса между полнотой документации и ее размера используется эвристика, ограничивающая деревья на длину веток и число веток в целом. Причем чем выше уровень вложенности символа b_i , тем сильнее ограничивается число веток, а на длину ограничение небольшое. Чем ниже уровень вложенности символа b_i , тем сильнее ограничивается длина веток, на число веток накладывается все меньшее ограничение.

Алгоритм построения дерева обязательных префиксов представлен в листинге 5.

Листинг 5 Построение дерева обязательных префиксов для документации

Require: Данные уровней вложенности L , максимальная глубина $d_{\max}$, ДКА сужения F , политика ограничений π

Ensure: Дерево обязательных префиксов T

```
1:  $\Lambda \leftarrow \pi.limits(0, d_{\max})$ 
2:  $T \leftarrow \text{ОбходВШирину}(F, \Lambda)$  ▷ очередь состояний
   ДКА; остановка при цикле на пути, петле Клини, превышении глубины ветки
   или суммарной степени ветвления
3: foreach переход  $(q, \sigma, q')$  на шаге обхода do
4:   сгруппировать символы с общей целью  $q'$  в сегмент  $\sigma \in \{t, s, (e), b_1, \dots\}$ 
5: end foreach
6:  $\text{ПостобработкаКорня}(T)$  ▷ добавить листья  $t, e, \varepsilon$  по флагам принятия и
   продолжения
7: if  $L = \emptyset$  then
8:   return  $\text{УсечьДерево}(T)$ 
9: end if
10:  $\mathcal{C} \leftarrow \emptyset$  ▷ кэш деревьев символов  $b_i$ 
11: foreach  $b \in \text{СимволыСкобок}(T)$  do
12:    $T_b \leftarrow \text{ДеревоСимвола}(L, b, 1, d_{\max}, \pi, \mathcal{C})$ 
13:    $\text{Склеить}(T, b, T_b)$  ▷ заменить ребро  $[b]$  вложенным деревом с границами (
     ...)
14: end foreach
15: повторять  $\text{СклеитьОставшиесяСкобки}(T, \mathcal{C})$ , пока в  $T$  есть символы  $b_i$ 
16: return  $\text{УсечьДерево}(T)$ 
```

Процедура ДеревоСимвола для символа b на уровне d строит ДКА языка, аннотированного символом b , выполняет для него ОбходВШирину с лимитами $\pi.limits(d, d_{\max})$, рекурсивно склеивает вложенные символы b_j и сохраняет результат в кэше $\mathcal{C}$.

Алгоритм извлечения обязательных инфиксов работает с ДКА как с графом, при этом кратные ребра схлопываются в одно. Псевдокод приведен в листинге 6. Алгоритм поиска мостов — алгоритм Тарьяна [7].

Листинг 6 Извлечение обязательных инффиксов

Require: Данные уровней L , ДКА шаблона A , ДКА сужения F , ограничения Λ

Ensure: Минимальное по включению множество строк обязательных инффиксов

M

```
1:  $M \leftarrow \emptyset$ 
2: foreach  $D \in \{A, F\}$  do
3:   if  $D$  слишком велик по  $\Lambda$  then
4:     перейти к следующему  $D$ 
5:   end if
6:    $G \leftarrow$  граф  $D$  с объединенными параллельными ребрами
7:    $\mathcal{B} \leftarrow \text{МостыТарьяна}(G)$ 
8:    $S \leftarrow \emptyset$ 
9:   foreach мост  $(u, v) \in \mathcal{B}$  do
10:    foreach направление  $(u', v') \in \{(u, v), (v, u)\}$  и символ  $\sigma$  на ребре
       $(u', v')$  do
11:      if ОбязательныйМост( $D, u', v', \sigma$ ) then
12:         $S \leftarrow S \cup \{(u', v', \sigma)\}$ 
13:      end if
14:    end foreach
15:  end foreach
16:   $\mathcal{C} \leftarrow \text{СклеитьЦепочки}(S)$   $\triangleright$  объединить последовательные шаги через
    состояние степени 2
17:  foreach цепочка  $c \in \mathcal{C}$  do
18:     $M \leftarrow M \cup \{\text{ФорматИнфикса}(L, D, c)\}$   $\triangleright$  токены  $s, (e), (\dots e)$  по
      наибольшему общему префиксу языка символа
19:  end foreach
20: end foreach
21: return МинимальныеИнфиксы( $M$ )  $\triangleright$  удалить инффиксы, являющиеся
    подстроками других
```

2.3 Пользовательский интерфейс

Пользовательский интерфейс обязан предоставлять пользователю возможность воспользоваться основными функциями модуля семантического анализатора, а именно решением задач распознавания перекрытий и извлечения документации к шаблонам функции.

Как уже было сказано, библиотеке сопутствует приложение с интерфейсом командной строки, использующее эту библиотеку. Это приложение позволяет пользователю указать файл с исходным кодом и выполнить одно из трех основных действий:

- решение упрощенной задачи распознавания перекрытий;
- решение полной задачи распознавания перекрытий;
- извлечение документации к шаблонам функции;

Для простоты взаимодействия с приложением весь вывод направляется в стандартный поток вывода в удобном для чтения формате. Таким образом, пользовательский интерфейс представляет собой интерфейс командной строки, позволяющий пользователю взаимодействовать с библиотекой с помощью указания флагов при запуске программы.

3 Технологическая часть

3.1 Обзор используемых технологий

Программный комплекс реализован на языке Rust [10]. Выбор обусловлен, с одной стороны, строгой системой типов и контролем над памятью, что важно при интенсивных операциях над автоматами, а с другой — выразительностью алгебраических типов данных (`enum`, `struct`), удобных для представления синтаксического дерева и абстрактных шаблонов. Стандартная библиотека `std` использовалась для контейнеров, строк, файлового ввода-вывода и многопоточности.

Архитектура проекта следует схеме, описанной в конструкторской части (рисунок 8): библиотека `diploma` содержит всю логику анализа, а бинарный модуль предоставляет интерфейс командной строки. Структура пакета приведена в листинге 6.

Листинг 6 — Корневой модуль библиотеки `diploma`

```
1  pub mod lexer;  
2  pub mod parser;  
3  pub mod sema;  
4  
5  pub mod types;  
6  pub mod issue_storage;  
7  pub mod utils;
```

Обработка исходного кода проходит три последовательных этапа: лексический анализ (`lexer`), синтаксический анализ (`parser`) и семантический анализ (`sema`). Первые два модуля преобразуют текст программы на Рефал-5λ в синтаксическое дерево; модуль `sema` содержит реализацию алгоритмов работы с абстрактными шаблонами, автоматами, распознавания перекрытий и извлечения документации.

Помимо основных модулей, библиотека включает вспомогательные компоненты. Модуль `types` определяет типы, необходимые для выдачи диагностических сообщений, (`Position`, `Diagnostic`, `Severity`), используемые лексером и парсером. Модуль `issue_storage` реализует обобщенное хранилище ошибок с типажом `Issue`.

Внешние зависимости проекта сведены к минимуму (таблица 2).

Таблица 2 — Внешние зависимости программного комплекса

Пакет	Назначение
regex	распознавание лексем в построчном лексере
smol_str	компактное хранение имен переменных и строковых литералов
fixedbitset	битовые множества финальных состояний ДКА и НКА
digits_iterator	форматирование индексов символов b_i в выводе

Корректность реализации проверяется модульными тестами, расположенными непосредственно в исходных файлах, тесты помечены (`#[cfg(test)]`). Дополнительно используется корпус тестовых программ `refal_test/`, включающий синтетические примеры и несколько сравнительно больших проектов на Рефал-5 и Рефал-5 λ : MSCP-A, SCP4, `refal5-lambda`.

3.2 Реализация модуля для работы с абстрактными шаблонами

Модуль `sema::abstract_pattern` реализует преобразование узлов синтаксического дерева в абстрактные шаблоны и построение по ним ДКА. Ключевые типы определены в файле `abstract_pattern.rs`; преобразование шаблона в автомат — в `to_dfa.rs`.

Центральной структурой является `AbstractPattern` — последовательность элементов `AbstractPatternElement`, соответствующих абстрактному шаблону из языка $\mathcal{W}_{\text{pat}}$. Перечисление элементов приведено в листинге 7.

Листинг 7 — Элементы абстрактного шаблона

```
1 pub enum AbstractPatternElement {
2     TVar,
3     SVar,
4     EVar,
5     Nested(AbstractPattern),
6     Constant(Constant),
7     AlternationRegex(Vec<u16>),
8 }
```

Варианты TVar, SVar и EVar соответствуют переменным t , s и e ; Nested — вложенному подшаблону в скобках; Constant — атомарному символу (символу, числу или строковой константе); AlternationRegex — появляется в промежуточных шагах преобразования шаблонов в ДКА и представляет собой некоторую альтернативу по части символов $b_1, b_2, \dots$. Причина появления констант в определении абстрактного шаблона будет рассмотрена позже.

При абстрактизации каждого предложения функции формируется структура PatternInfo, фиксирующая свойства исходного шаблона: наличие повторных переменных (had_repeated_variables), атомарных символов (had_constants), условий (had_condition). Эта информация используется при построении фактических языков и определяет, участвует ли шаблон в сужении шаблонов ниже.

Точка входа — функция abstractize_from_fn, обходящая тело функции и для каждого предложения вызывающая abstractize_left_part. Последняя преобразует левую часть предложения (LeftPart) в пару (AbstractPattern, PatternInfo). Внутри abstractize_pattern каждый терм шаблона (PatternTerm) отображается в элемент абстрактного шаблона; для переменных e выполняется слияние смежных вхождений (collapse_adjacent_evars), отражающее семантику конкатенации e -переменных в Рефале.

Алгоритм построения ДКА абстрактного шаблона (листинг 1) реализован функцией remove_nesting_with_levels в модуле screening. Функция используется также при извлечении документации.

Функция remove_nesting_with_levels обходит уровни вложенности от максимального к единице (в листинге 8). На каждом уровне собираются уникальные подшаблоны глубины d , для каждого строится ДКА, затем выполняется аннотированное объединение (Dfa::annotated_union_many). Полученные аннотации нумеруются и становятся символами нового алфавита; вложенные подшаблоны заменяются элементами AlternationRegex. Данные каждого уровня сохраняются в структуре LevelData для последующей склейки деревьев префиксов при генерации документации.

Листинг 8 — Цикл понижения максимальной глубины вложенности абстрактных шаблонов

```
1  for depth in (1..=max_depth).rev() {
2      let subpatterns = unique_ordered_subpatterns(&current, depth);
3      let pattern_dfas: Vec<Dfa> = subpatterns
4          .iter()
5          .map(|p| p.to_dfa(alphabet_size))
6          .collect();
7      let (_, annotation) = Dfa::annotated_union_many(&pattern_dfa_refs);
8      let vec_of_annotations = number_annotations(&annotation);
9      // замена Nested на AlternationRegex ...
10     alphabet_size = vec_of_annotations.len() as u16;
11 }
```

После понижения уровня вложенности (или для шаблонов без скобок) ДКА строится методом `AbstractPattern::to_dfa`. Каждый элемент переводится во фрагмент ДКА, фрагменты конкатенируются через `Dfa::concat_many`. Семантика фрагментов соответствует определению из конструкторской части: `TVar` — переход по любому символу алфавита, `SVar` — по символу *s*, `EVar` — итерация Клини над всем алфавитом, `AlternationRegex` — объединение переходов по указанным символам.

3.3 Реализация модуля для работы с автоматами

Подмодуль `sema::automata` содержит реализации ДКА и НКА, а также набор операций над ними. Представление ДКА в памяти — сжатое разреженное строковое (CSR), описанное в конструкторской части (рисунок 9).

Структура `Dfa` хранит начальное состояние, битовое множество финальных состояний, размер алфавита и четыре CSR-массива для исходящих и входящих ребер (в листинге 9). Отсутствующий переход интерпретируется как переход в неявное ловушечное состояние: метод `transition` возвращает `None`, что эквивалентно попаданию в непринимаящее мертвое состояние.

Листинг 9 — Структура ДКА и метод перехода

```

1  pub struct Dfa {
2      initial: u32,
3      finals: BitSet,
4      state_count: u32,
5      alphabet_size: u16,
6      out_offsets: Vec<u32>,
7      out_symbols: Vec<u16>,
8      out_targets: Vec<u32>,
9      in_offsets: Vec<u32>,
10     in_sources: Vec<u32>,
11     in_symbols: Vec<u16>,
12 }
13
14 pub fn transition(&self, s: u32, a: u16) -> Option<u32> {
15     match symbols.binary_search(&a) {
16         Ok(idx) => Some(self.out_targets[start + idx]),
17         Err(_) => None, // implicit trap
18     }
19 }

```

Все публичные операции, возвращающие ДКА, пропускают результат через `finalize`: удаление бесполезных состояний (`trim`), минимизация алгоритмом Хопкрофта [6] (`minimize`) и перенумерация состояний в диапазон $0 \dots n - 1$ с `initial = 0`.

Операции над автоматами и их назначение сведены в таблицу 3.

Таблица 3 — Операции над ДКА в модуле `automata`

Операция	Реализация	Применение
<code>union, intersection</code>	прямое произведение	построение сужений
<code>complement</code>	дополнение с явной ловушкой	сужение, <code>annotation_dfa</code>
<code>concat_many</code>	произведение конфигураций	построение ДКА шаблона
<code>annotated_union_many</code>	объединение кортежей состояний	раскрытие вложенности скобок
<code>minimize</code>	алгоритм Хопкрофта	канонизация после операций
<code>includes</code>	$\mathcal{L}(\text{other}) \subseteq \mathcal{L}(\text{self})$	распознавание перекрытий
<code>determinize</code>	построение подмножеств	детерминизация НКА

Проверка включения языков (`includes`) реализована через пересечение `other` с дополнением `self`: язык включен тогда и только тогда, когда пересечение пусто.

Аннотированное объединение (в листинге 10) возвращает пару (`Dfa`, `Annotation`). Тип `Annotation` хранит для каждого состояния объединения множество индексов операндов, в финальных состояниях которых автомат находится одновременно, а также флаг наличия полезного непринимаящего состояния (для введения символа $\emptyset$ в алфавит).

Листинг 10 — Аннотированное объединение ДКА

```
1 pub struct Annotation {
2     pub annotations: HashMap<u32, HashSet<u16>>,
3     pub has_non_final_state: bool,
4 }
5
6 pub fn annotated_union_many(dfas: &[&Dfa]) -> AnnotatedDfa { ... }
```

НКА используются исключительно как промежуточное представление при конкатенации через ε -переходы (`concat_to_nfa_many`) и немедленно детерминируются; в дальнейшем алгоритмах работа ведется только с ДКА.

3.4 Реализация алгоритма обнаружения перекрытий

Распознавание перекрытий реализовано в модуле `sema::screening`.

3.4.1 Упрощенная задача

Алгоритм из (листинг 2) и (листинг 3) реализован функцией `screened_patterns`, вызываемой через `screened_patterns_from_fn` для всей функции. Последовательность этапов следующая:

1. `abstractize_from_fn` — получение абстрактных шаблонов и `PatternInfo`;
2. `forget_constants` — замена атомарных символов на `SVar` (константы могут «искажать» язык при абстрактизации);
3. `remove_nesting` — раскрытие вложенности скобок;

4. построение ДКА для каждого шаблона;
 5. последовательная проверка включения языка в аккумулятор асс.
- Логика проверки приведена в листинге 11.

Листинг 11 — Распознавание перекрытых предложений (упрощенная задача)

```

1  fn screened_patterns(patterns: &[(AbstractPattern, PatternInfo)])
2      -> HashSet<u16>
3  {
4      let clean_patterns: Vec<_> = patterns
5          .iter().map(|p| p.0.forget_constants()).collect();
6      let (clean_patterns, alphabet_size) =
7          remove_nesting(&clean_patterns);
8      let dfas: Vec<Dfa> = clean_patterns.iter()
9          .map(|p| p.to_dfa(alphabet_size)).collect();
10
11     let mut acc = Dfa::of_empty_language(alphabet_size);
12     let mut result = HashSet::new();
13     for (i, (dfa, info)) in dfas.iter()
14         .zip(infos.iter()).enumerate()
15     {
16         if acc.includes(dfa) {
17             result.insert(i as u16);
18         }
19         if !info.had_repeated_variables
20             && !info.had_constants
21             && !info.had_condition
22         {
23             acc = acc.union(dfa);
24         }
25     }
26     result
27 }
```

В аккумулятор асс включаются языки только «чистых» шаблонов — тех, которые изначально удовлетворяли определению шаблонов $\mathcal{W}_{\text{pat}}$ и не содержат условий (таблица 4).

Таблица 4 — Условия участия шаблона в аккумуляторе асс

Флаг PatternInfo	Влияние
had_repeated_variables	шаблон не участвует в асс
had_constants	шаблон не участвует в асс
had_condition	шаблон не участвует в асс
все флаги ложны	язык шаблона добавляется в асс

3.4.2 Полная задача

Алгоритм из (листинг 4) реализован функцией `full_screened_patterns_from_fn`. Сначала вызывается упрощенная задача; затем для каждого перекрытого предложения i выполняется жадное удаление шаблонов p_j ($j < i$) сверху вниз: на каждом шаге из множества кандидатов удаляется p_j , после чего пересчитываются сужения; если язык i -го сужения остается пустым, удаление сохраняется. В результате для каждого перекрытого предложения получается минимальное по включению множество индексов предшествующих шаблонов, «ответственных» за перекрытие. Режим доступен пользователю через флаг `-full-screening` (подробнее — в разделе 3.7).

3.5 Реализация алгоритма извлечения документации

Извлечение документации реализовано в подмодуле `sema::docgen`, состоящем из пяти файлов: `mod.rs` (точка входа и политика ограничений), `prefix_tree.rs` (построение дерева префиксов), `glue.rs` (рекурсивная склейка деревьев символов b_i), `display.rs` (форматирование дерева префиксов в текстовую нотацию) и `infix.rs` (извлечение обязательных инфиксов).

Точка входа — функция `function_documentation(func, &DepthLimitPolicy)`, возвращающая для каждого предложения функции структуру `PatternDoc` с полями `index`, `screened`, `prefixes` и `mandatory_infixes`.

3.5.1 Подготовка шаблонов

Перед построением документации выполняется `prepare_patterns_for_documentation`: понижение уровня вложенности с

сохранением `LevelData` и, при необходимости, «забывание» содержимого глубоких скобок, если алфавит получается слишком большим. Если после понижения уровня вложенности размер алфавита достигает порога `MAX_DOC_ALPHABET = 50`, содержимое подшаблонов на текущей максимальной глубине заменяется на `(e)`, после чего понижение уровня вложенности повторяется.

3.5.2 Построение сужений и деревьев префиксов

Для каждого шаблона i вычисляется ДКА сужения: $\mathcal{L}(F_i) = \mathcal{L}(A_i) \setminus \bigcup_{j < i} \mathcal{L}(A_j)$, где объединение берется только по «чистым» шаблонам (аналогично алгоритму сужения). Если ДКА фактического языка пуст, предложение помечается как перекрытое. Иначе по сужению строится дерево обязательных префиксов.

Основной цикл в листинге 12.

Листинг 12 — Цикл извлечения документации для предложений функции

```

1  for (i, (dfa, info)) in dfas.iter()
2      .zip(infos.iter()).enumerate()
3  {
4      let residual = dfa.intersection(&acc.complement());
5      if residual.is_empty() {
6          out.push(PatternDoc { screened: true, ... });
7      } else {
8          let tree = build_documentation_prefix_tree(
9              &levels, max_depth, &residual, policy);
10         let prefixes = format_documentation_prefix_tree(&tree);
11         let mandatory_infixes =
12             mandatory_infixes(&levels, dfa, &residual, &infix_limits);
13         out.push(PatternDoc { screened: false, ... });
14     }
15     if !info.had_repeated_variables
16         && !info.had_constants && !info.had_condition
17     {
18         acc = acc.union(dfa);
19     }
20 }
```

Функция `build_prefix_tree` выполняет обход в ширину по ДКА, группируя переходы с общей целью в один сегмент дерева (символы `t`, `s`, `(e)` или `[b1 ...]`). Расширение останавливается при достижении пределов

PrefixTreeLimits: суммарной степени ветвления вдоль ветки и глубины ветки. Предел нужен для корректной обработки больших ДКА, иначе префиксное дерево слишком разрастается и может привести к зависанию программы.

Функция `build_documentation_prefix_tree (glue.rs)` рекурсивно «склеивает» деревья символов b_i , восстанавливая язык каждого символа через `annotation_dfa` и данные `LevelData`.

3.5.3 Обязательные инфиксы

Извлечение обязательных инффиксов (`infix.rs`) представляет ДКА как граф (кратные ребра схлопываются) и находит мосты — ребра, удаление которых увеличивает число компонент связности. Дополнительно проверяется, что ни одно финальное состояние не достижимо без прохождения «начала» моста (`is_mandatory_directed_bridge`). Множество найденных инффиксов фильтруется функцией `filter_minimal_infixes`, удаляющей инффиксы, являющиеся подстроками других.

3.5.4 Эвристики ограничения размера

Политика `DepthLimitPolicy` задает зависимость пределов степени ветвления и длины ветки от уровня вложенности (таблица 5). Чем глубже уровень (ближе к листьям скобочной структуры), тем сильнее ограничивается степень ветвления и слабее — длина ветки; к корню наблюдается обратная тенденция. Пользователь может переопределить базовые значения через флаги `-fanout` и `-depth` (см. таблицу 5).

Метод `DepthLimitPolicy::limits(depth, max_depth)` вычисляет конкретные `PrefixTreeLimits` для заданного уровня вложенности.

3.6 Схема работы семантического анализатора

Итак, после описания всех модулей и алгоритмов, для общей картины на рисунке 10 представлена схема работы семантического анализатора.

Таблица 5 — Параметры DepthLimitPolicy и соответствующие флаги командной строки

Поле	Искомое значение	Флаг	Смысл
base_fanout	4	-fanout	степень ветвления на глубочайшем уровне
fanout_step	2	—	приращение степени ветвления на уровень вверх
deepest_length	8	-depth	длина ветки на глубочайшем уровне
length_step	2	—	уменьшение длины на уровень вверх
min_length	2	—	минимальная длина ветки



Рис. 10 — Схема работы подсистемы семантического анализатора.

Обе ветви опираются на общий этап абстрагирования и на операции модуля automata. Ветвь screening решает задачу распознавания перекрытий: сначала в упрощенном режиме фиксируются лишь номера экранируемых предложений, затем в полном режиме для каждого из них вычисляется минимальное

множество предшествующих шаблонов. Ветвь `docgen` для неперекрытых предложений строит человекочитаемое описание языка шаблона в виде обязательных префиксов и инфиксов; ограничения размера документации задаются политикой `DepthLimitPolicy`.

3.7 Руководство пользователя

3.7.1 Получение и сборка

Для сборки необходим установленный компилятор Rust (рекомендуется стабильная версия). Из корневого каталога проекта выполните:

```
cargo build --release
```

Исполняемый файл будет помещен в каталог `target/release/` и называться `diploma`.

3.7.2 Запуск

Приложение принимает путь к файлу с исходным кодом на Рефал-5λ и один из трех режимов работы. Синтаксис вызова:

```
diploma --basic-screening <file.ref>
diploma --full-screening <file.ref>
diploma --doc <file.ref> [--fanout N] [--depth N]
```

Режим `-basic-screening` решает упрощенную задачу распознавания перекрытий и выводит номера перекрытых предложений для каждой функции файла. Режим `-full-screening` решает полную задачу и дополнительно указывает минимальные множества предшествующих шаблонов, вызвавших перекрытие. Режим `-doc` извлекает документацию: для каждого неперекрытого предложения — список обязательных префиксов и, при наличии, обязательных инфиксов. Параметры `-fanout` и `-depth` управляют эвристиками ограничения размера деревьев префиксов (см. таблицу 5).

3.7.3 Пример использования

Рассмотрим функцию `h` из в листинге 13.

Листинг 13 — Пример функции с экранированием

```
1  h {  
2    e.x s.1 s.2 e.y = ...;  
3    e.x (e.3) s.4 e.y = ...;  
4    e.x (e.5) = ...;  
5    e.x s.6 (e.7) e.y = ...;  
6  }
```

Положим, эта функция находится в файле `example.ref`. Тогда запуск `diploma -basic-screening example.ref` даст следующий результат:

```
Function h - WARNING:  
sentence #4 is screened
```

Рассмотрим функцию `y` из в листинге 14.

Листинг 14 — Пример функции с нетривиальным сужением последнего шаблона

```
1  y {  
2    (e.1 s.1 e.2 s.2 e.3 t.3) e.4 = ...;  
3    (e.1 () (t.1 e.3) e.4) e.5 t.2 = ...;  
4    (t.1 t.2 t.3) e.1 = ...;  
5    (e.1 (e.2) e.3 () e.4) e.5 = ...;  
6    (e.1 t.1 s.2 t.3 e.2 s.4) = ...;  
7  }  
8
```

Положим, эта функция находится в файле `example.ref`. Тогда запуск `diploma -doc example.ref` даст следующий результат для пятого шаблона функции:

...

Pattern 5 is a subpattern of either:

| (() (t e) t e) |

| (() s (t e) t e) |

| ((t e) t e) |

При успешном выполнении программа возвращает код 0; при ошибке чтения файла, синтаксического разбора или некорректных аргументов — код 1.

4 Аналитическая часть

Анализ проводился на том же корпусе программ, что указан в конструкторской части. Одна и та же статистика собиралась с разного подмножества функций. В первый раз в статистике учитывались все функции корпуса полностью. В другой раз были учтены только «хорошие» функции, у которых половина и более шаблонов вносят вклад в сужение шаблонов ниже.

Размер ДКА и алфавитов — это ключевые параметры, влияющие на производительность алгоритмов. Начнем с анализа ДКА. Среди них наибольший интерес представляют ДКА сужений, так как именно с ними выполняется наибольшее число различных. Стоит отметить, что размер самих ДКА 0-шаблонов не так интересен, т.к. число состояний в минимальном ДКА шаблона всегда не больше его длины. Гистограммы числа состояний сужений для всех функций корпуса и для «хороших» функций представлены на рисунках 11 и 12 соответственно.

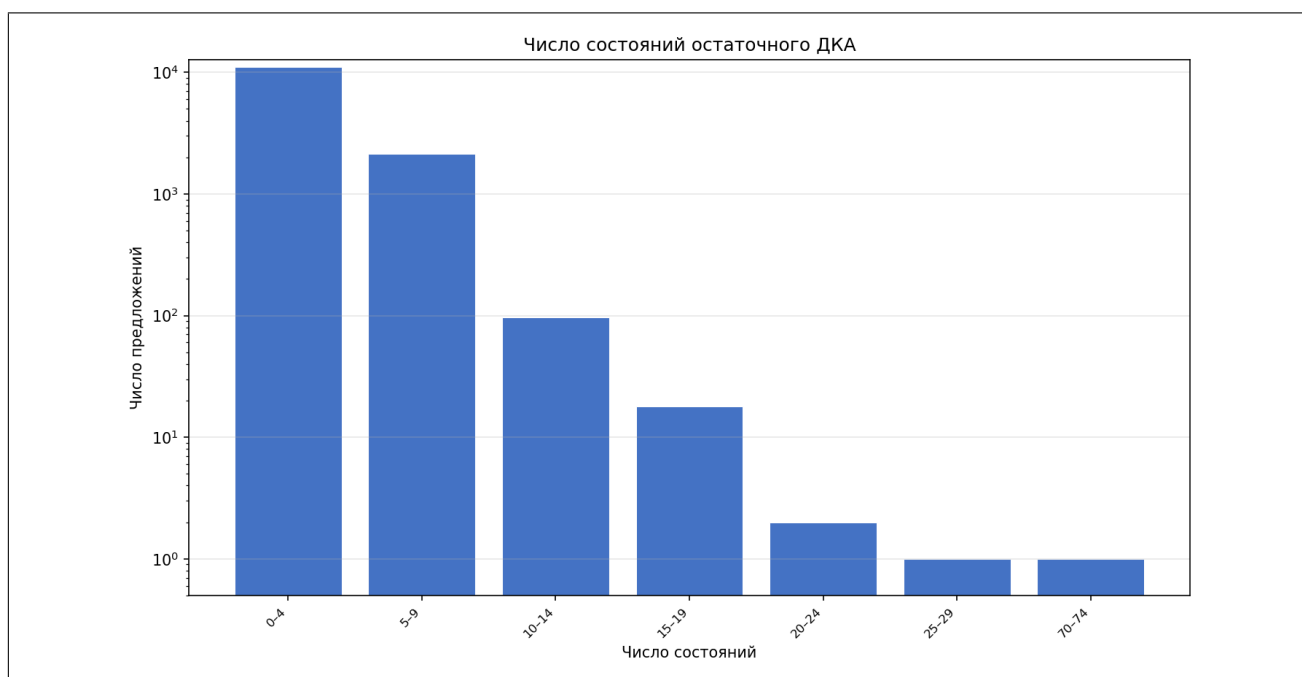


Рис. 11 — Гистограмма числа состояний остаточного ДКА (все функции корпуса).

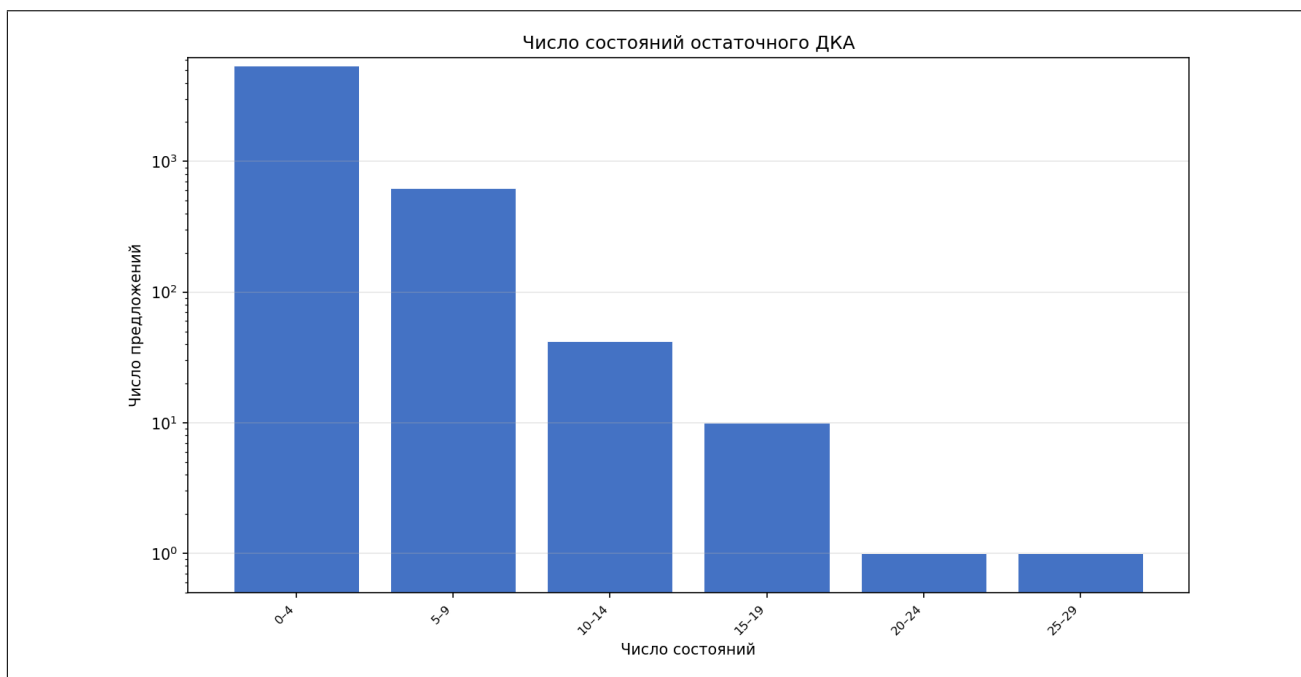


Рис. 12 — Гистограмма числа состояний остаточного ДКА («хорошие» функции).

Для удобства визуального восприятия используется логарифмическая шкала по вертикальной оси: абсолютное большинство (более 99%) ДКА сужений имеют не более 20 состояний. Схожая форма гистограммы наблюдается для обеих множеств функций, причем наибольшая разница между гистограммами наблюдается слева. Это обусловлено тем, что большие ДКА фактического языка порождают преимущественно функции, большая часть шаблонов в которых вносит вклад в сужение языка, а для функций, в которых, например, много условий, размеры ДКА сужений совпадают меньше.

На правом конце гистограммы для всех функций виден исключительный случай — один ДКА с размером 70-74 состояния. Этот выброс относится к функции `ResetFuncName` из проекта SCP4, в частности, к ее первому предложению, вид которого приведен в листинге 15.

Листинг 15 — Часть функции ResetFuncName из проекта SCP4

```
1 ResetFuncName {
2   s.newName s.old s.new '/* ** This comments is needed for Graph- to C-
   ↳ code of transforme ** */' = ...;
3   s.newName s.old s.new 0 = ...;
4   s.newName s.old s.new e.1 0 = ...;
5   s.newName s.old s.new e.1 = ...;
6 }
```

Из-за последовательности констант в конце шаблона первого предложения, сужение автомата шаблона, совпадающее в данном случае с самим ДКА шаблона, имеет по состоянию на каждый символ последовательности в кавычках, что и приводит к такому размеру ДКА.

Гистограммы размера алфавита ДКА 0-шаблонов для всех функций корпуса и для «хороших» функций представлены на рисунках 13 и 14 соответственно.

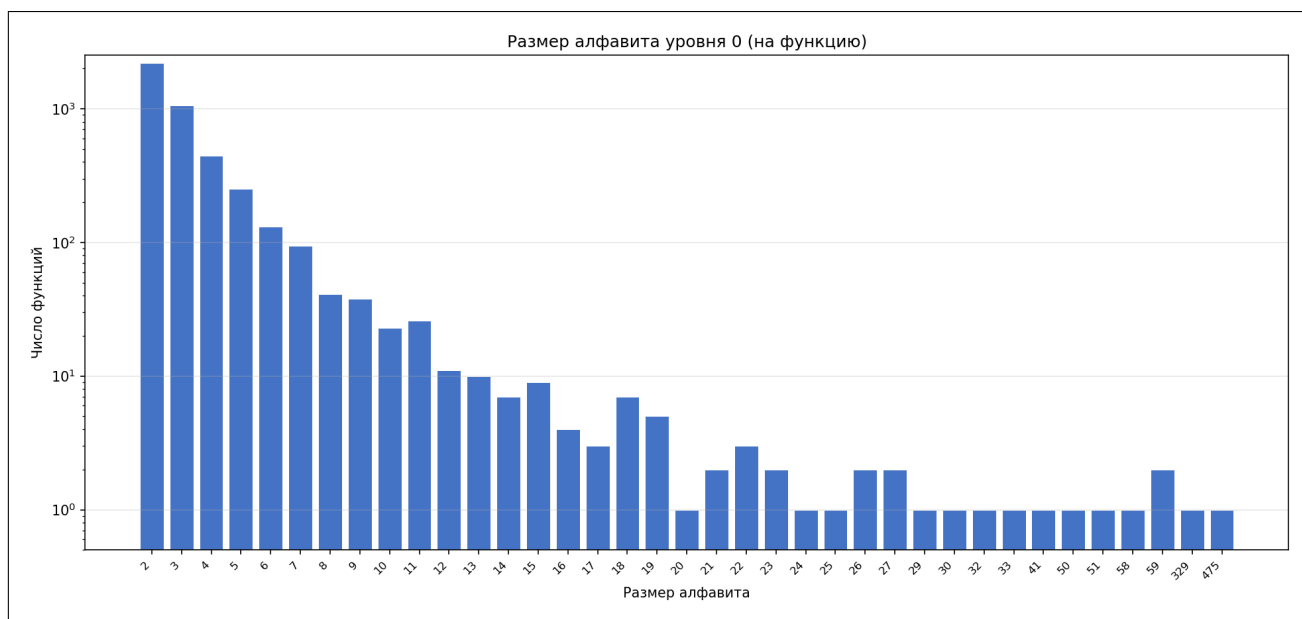


Рис. 13 — Гистограмма размера алфавита ДКА 0-шаблонов (все функции корпуса).

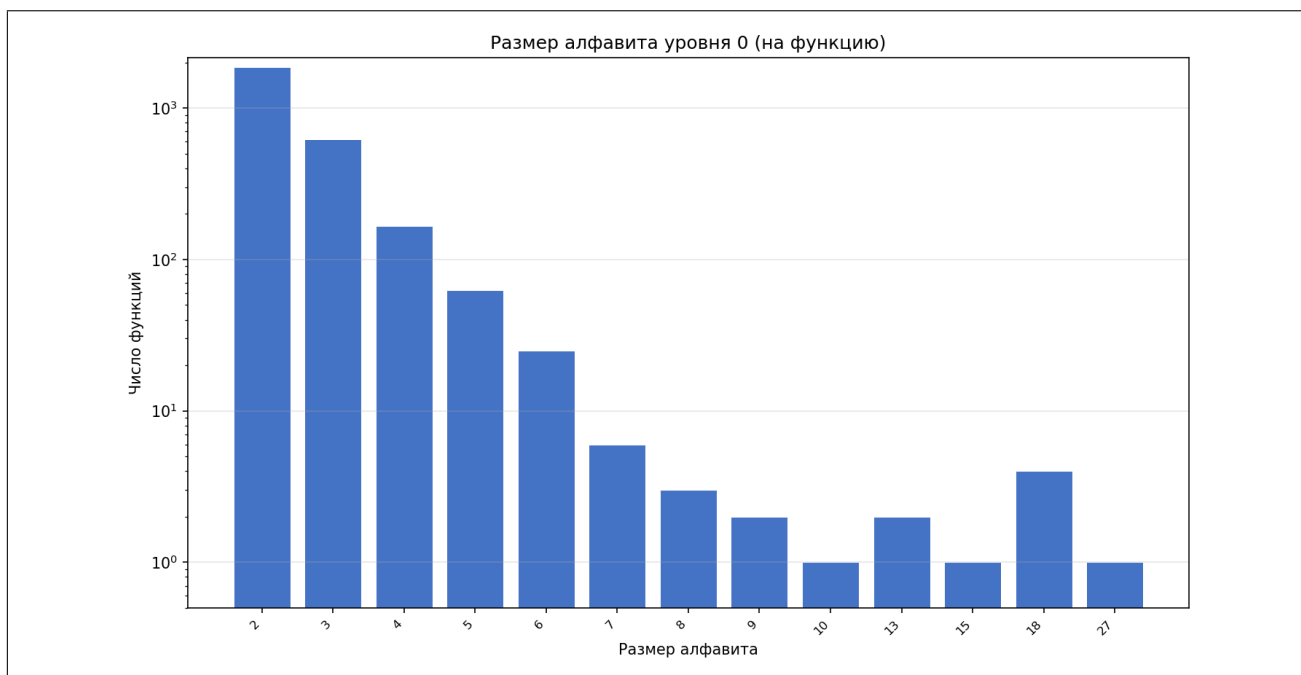


Рис. 14 — Гистограмма размера алфавита ДКА 0-шаблонов («хорошие» функции).

Наблюдается картина, аналогичная гистограмме числа состояний остаточного ДКА: абсолютное большинство (более 99%) ДКА 0-шаблонов имеют алфавит размером не более 20 символов. Выбросы по числу символов на правом конце гистограммы наблюдаются для сложных функций, в которых много различных 1-шаблонов.

4.1 Тест производительности

Целью теста является оценка скорости работы реализованных режимов распознавания перекрытий шаблонов: упрощенного (-basic-screening) и полного (-full-screening). Измерялось суммарное время семантического анализа всех функций корпуса; синтаксический разбор в замеры не входил.

4.1.1 Конфигурация стенда

Замеры выполнялись на рабочей станции со следующими характеристиками центрального процессора (фрагмент вывода команды `lscpu`):

```
Architecture:      x86_64
CPU(s):            8
```

Model name: 11th Gen Intel(R) Core(TM) i5-11300H @ 3.10GHz
Thread(s) per core: 2
Core(s) per socket: 4
CPU max MHz: 4400.0000
L1d cache: 192 KiB (4 instances)
L2 cache: 5 MiB (4 instances)
L3 cache: 8 MiB (1 instance)

Программный комплекс собирался в режиме release (cargo build -release) компилятором Rust 1.91. Для автоматизации замеров использовалась утилита screening_bench -bench из состава проекта.

4.1.2 Методика измерений

В качестве корпуса брались все файлы *.ref из каталога refal_test/real_projects/. Из проекта Рефал-5λ исключался лишь каталог autotests; остальной исходный код компилятора и сопутствующих утилит входил в корпус. В него вошли MSCP-A (47 файлов), SCP4 (37 файлов) и Рефал-5λ без тестов (79 файлов), всего 163 файла и 4820 функций.

На каждой итерации фиксировалось суммарное время прохода по всему корпусу отдельно для режимов -basic-screening и -full-screening. Перед серией измерений выполнялся прогрев: 10 полных проходов по корпусу для каждого режима. Рабочая выборка составляла 100 итераций. Доверительные интервалы уровня 0,95 для средних значений строились по распределению Стьюдента:

$$\bar{x} \pm t_{0,975, n-1} \cdot \frac{s}{\sqrt{n}}, \quad n = 100,$$

где s — выборочное стандартное отклонение времени одного прохода по корпусу. Дополнительно после прогрева для каждой функции выполнялся одиночный замер; по ним составлялся рейтинг наиболее затратных функций.

4.1.3 Результаты

Сводные результаты приведены в таблице 6.

Таблица 6 — Суммарное время распознавания перекрытий на корпусе (секунды, $n = 100$; s — выборочное стандартное отклонение)

Режим	Среднее	ДИ 0,95	s
-basic-screening	2,412	[2,384; 2,440]	0,141
-full-screening	2,394	[2,366; 2,422]	0,140

Среднее время обработки одной функции на контрольном проходе составило 0,46 мс (режим -basic-screening), медиана — 0,09 мс, 95-й перцентиль — 1,11 мс. Для режима -full-screening значения близки (среднее 0,45 мс, медиана 0,08 мс), что объясняется малым числом функций с перекрытиями: полный режим выполняет дополнительную работу лишь для экранируемых предложений.

Пятнадцать наиболее затратных функций по времени -basic-screening приведены в таблице 7. В сумме они дают около 40% времени контрольного прохода по корпусу.

Таблица 7 — Наиболее затратные функции корпуса (контрольный проход; $|\Gamma_0|$ — размер алфавита ДКА 0-шаблонов, $\Sigma_{\text{сост.}}$ — сумма чисел состояний ДКА 0-шаблонов по предложениям)

Проект	Функция	t_{basic} , мс	t_{full} , мс	Предл.	$ \Gamma_0 $	$\Sigma_{\text{сост.}}$
Рефал-5 λ	DoGenSubst (GenSubst-Simple)	221	218	37	59	242
Рефал-5 λ	DoGenSubst (GenSubst-Save)	219	219	37	59	242
SCP4	Red	139	141	44	475	175
Рефал-5 λ	Solve-SymmClashes-Aux	83	85	53	58	264
SCP4	RigitMatch	57	57	27	329	80
SCP4	CCDRT	36	36	98	17	810
SCP4	TAWAYL	31	30	37	22	222
SCP4	APPL1	23	23	35	22	166
MSCP-A	DeleteDoubleEqs	17	17	3	4	6
Рефал-5 λ	Solve-Clashes	17	17	31	19	153
SCP4	UnBr	13	13	11	33	37
Рефал-5 λ	Decompile-Pattern-Hole	12	13	31	16	219
MSCP-A	GenerateNextLevel	11	11	7	33	84
SCP4	MST-Parse	10	10	48	15	217
MSCP-A	WeightMGU	10	10	23	24	68

4.1.4 Анализ результатов

Распределение времени обработки крайне неоднородно: подавляющее большинство функций укладывается в доли миллисекунды, тогда как единичные выбросы задают суммарное время прохода по корпусу. Различие между режимами `-basic-screening` и `-full-screening` на уровне корпуса статистически незначимо: доверительные интервалы перекрываются, отношение средних времен близко к единице. Это согласуется с редкостью перекрытий в реальных программах: для функций из таблицы 7 число экранируемых предложений равно нулю, и полный режим не выполняет дополнительных пересчетов сужений.

Наибольший вклад вносят функции компилятора Рефал-5λ. Две первые строки таблицы 7 соответствуют одноименной вспомогательной функции `DoGenSubst` из модулей `GenSubst-Simple` и `GenSubst-Save` (файлы `HighLevelRASL-GenSubst-Simple.ref` и `HighLevelRASL-GenSubst-Save.ref`). Это два варианта генератора подстановок при переводе RASL-программы: `GenSubst-Simple` вызывается на пути без оптимизации результата (`NoOpt`), `GenSubst-Save` — на оптимизированном пути (`OptResult`), где дополнительно формируются «сохранители» значений переменных. Наборы предложений в обеих функциях `DoGenSubst` совпадают по структуре, поэтому время их обработки близко; основная стоимость — в последовательном построении и объединении автоматов для 37 предложений при алфавите из 59 символов. Аналогично, `Solve-SymmClashes-Aux` и `Solve-Clashes` — функции с десятками предложений из модуля сопоставления шаблонов компилятора.

Среди проектов SCP4 выделяется функция `Red`: при 44 предложениях алфавит 0-шаблонов достигает 475 символов из-за длинной последовательности констант в шаблоне (аналогично случаю `ResetFuncName`, обсуждаемому выше). Функция `RigitMatch` демонстрирует сходную картину с алфавитом 329 символов при меньшем числе предложений.

Функция `CCDRT` из SCP4 иллюстрирует иной тип затратности: алфавит невелик (17 символов), но функция содержит 98 предложений, а суммарное число состояний ДКА 0-шаблонов превышает 800. Здесь доминирует линейная по числу предложений стоимость построения автоматов, а не размер алфавита.

В целом время обработки функции определяется произведением числа предложений, размера алфавита 0-шаблонов и сложности вложенной структуры шаблонов. Для типичных функций корпуса (1–3 предложения, алфавит из нескольких символов) анализ занимает менее 0,1 мс; практическая пригодность инструмента ограничена лишь отдельными «тяжелыми» функциями с десятками предложений или широким алфавитом.

ЗАКЛЮЧЕНИЕ

В рамках работы были выполнены поставленные задачи — порождение документации и распознавание перекрытий шаблонов. Был разработан и реализован алгоритм получения ДКА по шаблонам. Написаны алгоритмы для работы с этими ДКА, решающие задачи извлечения документации и распознавания перекрытий шаблонов. Произведено тестирование на синтетических программах и на реальных проектах. Тесты производительности показали высокую эффективность программы, что позволяет использовать ее при разработке реальных Рефал-проектов.

Дальнейшем развитием данной работы может послужить, например, более глубокий анализ ДКА сужений или внедрение в алфавит атомарных символов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Турчин В. Ф. Рефал — теория, техника, практика. — 2-е изд. — Москва : Наука, 1989.
2. Graf S., Peyton Jones S., Scott R. G. Lower Your Guards: A Compositional Pattern-Match Coverage Checker // Proceedings of the ACM on Programming Languages. — 2020. — Т. 4, ICFP. — 107:1—107:30. — DOI: 10.1145/3408989. — URL: <https://doi.org/10.1145/3408989>.
3. Dotta M., Suter P., Kuncak V. On Static Analysis for Expressive Pattern Matching [Электронный ресурс] : тех. отч. / École polytechnique fédérale de Lausanne (EPFL). — Lausanne, Switzerland, 2008. — LARA-REPORT-2008—004. — URL: <https://infoscience.epfl.ch/record/119123> (дата обр. 28.06.2026).
4. The Rust Project. `rustc_pattern_analysis::usefulness` [Электронный ресурс]. — URL: https://doc.rust-lang.org/nightly/nightly-rustc/rustc_pattern_analysis/usefulness/index.html (дата обр. 28.06.2026) ; Internal documentation of the Rust compiler; usefulness and reachability analysis for pattern matching.
5. On Cascades of Reset Automata / R. Borelli [и др.] // 42nd International Symposium on Theoretical Aspects of Computer Science (STACS 2025). — Dagstuhl, Germany : Schloss Dagstuhl — Leibniz-Zentrum für Informatik, 2025. — (Leibniz International Proceedings in Informatics (LIPIcs)). — DOI: 10.4230/LIPIcs.STACS.2025.20. — URL: <https://doi.org/10.4230/LIPIcs.STACS.2025.20>.
6. Hopcroft J. E., Motwani R., Ullman J. D. Introduction to Automata Theory, Languages, and Computation. — 3-е изд. — Addison-Wesley, 2006.
7. Tarjan R. E. A Note on Finding the Bridges of a Graph // Information Processing Letters. — 1974. — Т. 2, № 6. — С. 160—161. — DOI: 10.1016/0020-0190(74)90003-9.
8. Compilers: Principles, Techniques, and Tools / A. V. Aho [и др.]. — 2-е изд. — Addison-Wesley, 2006.

9. Рефал-5λ. Приложение В. Краткий справочник [Электронный ресурс] / Кафедра информатики и систем управления МГТУ им. Н. Э. Баумана. — URL: <https://bmstu-iu9.github.io/refal-5-lambda/B-reference.html> (дата обр. 27.06.2026).
10. The Rust Project Developers. The Rust Standard Library [Электронный ресурс]. — URL: <https://doc.rust-lang.org/stable/std/index.html> (дата обр. 28.06.2026).